



Fine-tuning de Modelos de Lenguaje Grandes (LLMs) para las consultas sobre fuentes de datos abiertas

Grado en Ingeniería Informática

Trabajo Fin de Grado

Autor:
Iván Guillamón Baños
Tutor/es:
Aurora González Vidal
Antonio Martínez Ibarra
2026-06-30



Facultad
Informática
Universidad
Murcia

Fine-tuning de Modelos de Lenguaje Grandes (LLMs) para las consultas sobre fuentes de datos abiertas

---

Diseño y evaluación de un sistema híbrido SQL+RAG con modelos de código abierto sobre infraestructura local

Autor
Iván Guillamón Baños

Tutor/es
Aurora González Vidal
Antonio Martínez Ibarra

Facultad de Informática

Grado en Ingeniería Informática

UNIVERSIDAD DE MURCIA

Murcia, 2026-06-30

Preámbulo

El interés por los modelos de lenguaje de gran tamaño y por las herramientas que permiten construir aplicaciones reales a partir de ellos surgió durante los últimos cursos del Grado. El campo evoluciona a un ritmo extraordinario y, sin embargo, una parte considerable de los desarrollos publicados se apoya en servicios en la nube y modelos comerciales de gran tamaño, dejando relativamente menos explorado el escenario en el que todo el sistema debe ejecutarse de forma local con recursos limitados. Este Trabajo de Fin de Grado parte de esa premisa para explorar hasta dónde es posible llegar combinando dos paradigmas complementarios —la generación de SQL a partir de lenguaje natural y la generación aumentada por recuperación— en un único sistema híbrido, evaluado de forma sistemática sobre un caso de uso real.

La elección del ecosistema del Mar Menor como caso de estudio responde a su condición de problema cercano y bien documentado, sobre el que conviven extensas series temporales de monitorización ambiental y un cuerpo de literatura científica que abarca tanto estudios específicos de la laguna como fundamentos más amplios de las ciencias marinas. Esta convivencia lo convierte en un escenario especialmente adecuado para someter a prueba un sistema cuya razón de ser es precisamente la integración de ambas fuentes de información.

El trabajo se ha desarrollado en el marco del Grado en Ingeniería Informática de la Universidad de Murcia, bajo la tutela de Aurora González Vidal y Antonio Martínez Ibarra.

Agradecimientos

A Aurora González Vidal y Antonio Martínez Ibarra, por su implicación en este proyecto a lo largo de todas sus fases. Su disponibilidad constante para resolver dudas, la rapidez con la que han respondido a cada consulta y su predisposición a ayudar siempre que se lo he pedido han sido fundamentales para llevar a buen puerto el trabajo. Agradezco también la libertad que me han dado para tomar decisiones técnicas y explorar el problema desde mi propio enfoque, sin dejar por ello de orientarme cuando lo he necesitado.

A mi familia, por el apoyo incondicional durante todos estos años de Grado y, en especial, durante los meses de desarrollo de este Trabajo Fin de Grado.

A mis compañeros de carrera, por las discusiones compartidas, las dudas resueltas en común y los buenos momentos que han hecho mucho más llevadero este tramo final.

Resumen

La adopción de los Large Language Models (LLMs) ha transformado la forma en que los usuarios pueden interactuar con sistemas informáticos complejos, ya que permite formular preguntas en lenguaje natural sin necesidad de dominar dialectos de consulta específicos. Sin embargo, estos modelos por sí solos no pueden acceder de forma fiable a fuentes externas, lo que hace que generen respuestas plausibles pero infundadas, un fenómeno conocido como alucinación. Para mitigar esta limitación se han consolidado dos enfoques complementarios: la generación de Structured Query Language (SQL) a partir de lenguaje natural (*Text-to-SQL*), que permite consultar bases de datos estructuradas, y la generación aumentada por recuperación (Retrieval-Augmented Generation (RAG)), que fundamenta la respuesta del modelo en fragmentos extraídos de un corpus documental. Ahora bien, ninguno de los dos enfoques es suficiente por sí mismo en escenarios reales, donde los datos cuantitativos y la literatura científica son frecuentemente complementarios.

Este Trabajo Fin de Grado aborda este problema mediante el diseño, la implementación y la evaluación experimental de un sistema híbrido que integra ambos enfoques en un único *pipeline* ejecutado íntegramente sobre infraestructura local. El sistema se articula en cuatro componentes desacoplados: un clasificador de intención basado en similitud semántica sobre *embeddings*, que enruta cada consulta sin recurrir a inferencia generativa; un módulo SQL con guardarraíles de seguridad y un bucle de autocorrección guiado por los errores del motor de base de datos; un módulo RAG con un *prompt* orientado a la fundamentación en la evidencia recuperada; y un orquestador que centraliza la generación de la respuesta final. La validación del sistema toma como caso de uso empírico el ecosistema del Mar Menor, un escenario que dispone de extensas series temporales de monitorización ambiental. Para la recuperación documental, se ha integrado un corpus consolidado de literatura científica que abarca tanto estudios específicos de la laguna como fundamentos más amplios de las ciencias marinas, lo que convierte a este dominio en un entorno especialmente adecuado para someter a prueba un sistema híbrido de esta naturaleza.

Los resultados experimentales son alentadores: el clasificador de intención alcanza un 92.4 % de exactitud global, el módulo RAG produce respuestas funda-

mentadas hasta en el 90 % de los casos sobre preguntas conceptuales, y el análisis de fallos del módulo SQL permite caracterizar con precisión los límites de un modelo de 8B parámetros en este escenario. El trabajo demuestra la viabilidad de un sistema híbrido SQL+RAG construido con modelos de código abierto de tamaño moderado, y proporciona un marco metodológico y experimental diseñado para facilitar futuras ampliaciones del sistema, como la recuperación híbrida densa-léxica, la evaluación de modelos de lenguaje de mayor escala para el razonamiento lógico, o la validación humana de los resultados.

Extended Abstract

The widespread adoption of LLMs has transformed the way users interact with complex information systems. What used to require knowledge of a specific query dialect or navigation through structured interfaces can now be expressed through questions formulated in natural language, without the user having to master the technical details of the underlying information source. This shift is particularly relevant in technical and scientific domains, where useful information tends to be scattered across heterogeneous repositories that are typically accessible only to specialised users.

However, LLMs alone do not solve the underlying problem. Their knowledge is fixed at the moment of training, and they lack reliable mechanisms to access external sources. When queried about specific data or particular publications, they tend to produce plausible but unfounded responses, a phenomenon known as hallucination. To mitigate this limitation, two complementary but distinct approaches have consolidated in the literature. The first is the generation of SQL queries over structured databases from the user's question, known as Text-to-SQL. The second is RAG, which retrieves relevant fragments from a documentary corpus to ground the model's response. Both paradigms have reached a considerable level of maturity and have been incorporated into a growing number of real-world applications.

Neither approach, however, is sufficient on its own in many practical scenarios. Quantitative data cannot be interpreted without the theoretical framework that gives them meaning, and scientific literature can hardly substitute empirical measurements when the question demands a concrete figure. Hence the need for hybrid systems capable of combining both sources and deciding, for each query, which components should intervene and how the final result is synthesised. The integration of both paradigms in a single pipeline with automatic routing remains an open problem, particularly under the constraint of operating entirely on local infrastructure with open-source models, a restriction that in many real settings is imposed by privacy, cost or reproducibility concerns.

This Final Degree Project addresses this problem through the design, implementation and experimental evaluation of a hybrid SQL+RAG system that operates entirely on local resources. The general objective is to demonstrate that a hybrid

architecture combining structured querying and document retrieval can be built and operated locally with open-source models of moderate size, while preserving the quality guarantees expected of each paradigm individually.

This general objective is broken down into four specific objectives. First, to design an efficient intent classifier that autonomously decides the route each query must follow, avoiding the use of a full LLM for a routing task. Second, to implement a SQL module capable of generating correct queries over the real schema of the database and of recovering from the errors that occur during generation. Third, to implement a RAG module that retrieves relevant fragments from a documentary corpus and minimises hallucination by means of a prompt oriented to grounding on the retrieved evidence. Fourth, to integrate the three previous components through an orchestrator that centralises the generation of the final response, ensuring that both data and literature reach the model without intermediate reinterpretations that might compromise their fidelity.

The validation of this architecture has been carried out on the case study of the Mar Menor, a hypersaline coastal lagoon in southeastern Spain. The choice is justified by two characteristics that make it particularly appropriate: the existence of a heterogeneous network of monitoring sensors that produces high-resolution time series, and the availability of a corpus of scientific literature encompassing both Mar Menor-specific studies and broader marine science publications. The coexistence of both bodies of information in the same domain makes the Mar Menor a representative scenario of the kind of integrated query problem that motivates the work.

The system is organised as a four-stage pipeline coordinated by an orchestrator. A natural-language question is first received by the orchestrator and delegated to an intent classifier, which determines whether the answer requires querying the empirical database (`SQL` route), retrieving fragments from the scientific corpus (`RAG` route), or synthesising both sources (`HIBRIDO` route). The corresponding module is then invoked and its raw output, a DataFrame in the case of SQL or a set of retrieved fragments in the case of RAG, is returned to the orchestrator, which generates the final response.

The intent classifier is based on semantic similarity rather than on generative inference. A bank of categorised examples is embedded at startup using the `multilingual-e5-small` model, and each incoming query is encoded and compared against this bank. The category whose top-k examples yield the highest average cosine similarity is selected. A tie-detection margin allows ambiguous queries to

be routed explicitly to the `HIBRIDO` category. This decision, averaging the top-k per class rather than counting votes in a global K-Nearest Neighbors (K-NN), was hypothesised at design time and confirmed empirically to outperform majority voting across all values of k considered.

The SQL module uses Llama 3 with a carefully engineered prompt that combines role assignment (an Online Analytical Processing (OLAP) data engineer specialised in DuckDB), chain-of-thought prompting through an explicit `<thought_process>` block, and few-shot examples covering the main query patterns of the domain. The schema of the database is injected verbatim to prevent hallucination of non-existent columns. The output is processed by an autocorrection loop that handles three failure modes (truncated responses, unparseable output, and DuckDB execution errors) and feeds each one back to the model in a way that triggers a new generation under the same deterministic temperature configuration. A security layer parses the generated query into lexical tokens and ensures that it is strictly read-only, rejecting any statement that contains destructive operations.

The RAG module indexes a corpus of 36 scientific documents in PDF format by splitting them into 1000-character chunks with a 200-character overlap, vectorising each chunk with the `multilingual-e5-small` model under the `passage:` prefix, and persisting the resulting embeddings in ChromaDB. At query time, the user question is encoded under the `query:` prefix and used to retrieve the top $2K$ candidates (over-fetching), which are then filtered by a cosine distance threshold and deduplicated. In addition, a residual filter discards chunks dominated by bibliographic references.

The orchestrator centralises the final response generation. Three modes are supported. In SQL mode, the DataFrame is inspected and classified into one of four shapes (aggregation, time series, small table, large table), each triggering a specific instruction in the LLM prompt. In RAG mode, the prompt enforces grounding to the retrieved context through structural delimiters, an explicit restriction to use no prior knowledge, and an abstention template for cases without supporting evidence. In hybrid mode, both sources are injected together and the model is instructed to synthesise them. Under the system's graceful degradation policy, if the SQL module fails irrecoverably during a hybrid query, the orchestrator falls back to RAG mode and prepends an explicit warning informing the user that empirical evidence is absent.

The evaluation is organised in three parts. For the intent classifier, a balanced test set of 66 questions (22 per class) was constructed independently from the

example bank, covering the three categories the classifier must distinguish: `SQL` for questions that should be answered from the empirical database, `RAG` for questions that should be answered from the scientific corpus, and `HIBRIDO` for questions that require synthesising both sources. Five decision strategies were compared (random baseline, 1-NN, mean over all examples per class, mean over top-k per class, and majority voting over top-k), each evaluated with and without the tie margin, and over two example bank sizes (21 and 63 examples). The configuration deployed in the system, mean top-4 per class with the tie margin, achieves 92.4 % accuracy and per-class F1 scores above 90 %. Suppressing the tie margin degrades global accuracy to 90.9 %, with the `SQL` route absorbing a disproportionate share of the errors. Reducing the example bank to 21 entries collapses accuracy to 72.7 % and severely impacts the `HIBRIDO` class, whose recall drops to 59.1 %. These results jointly validate the two key design decisions of the classifier, namely the averaging strategy and the tie margin.

For the RAG module, an LLM-as-judge protocol was implemented. The judge evaluates each response along two complementary dimensions on a 1-to-5 scale: Context Relevance, which captures whether the retrieved fragments are useful for answering the question, and Groundedness, which captures whether the generated answer remains faithful to the retrieved evidence. The experiment swept the full Cartesian product of $K \in \{3, 5, 7\}$ and ten threshold values, applied to two test sets of 30 questions each, totalling 1800 evaluations. The optimal configuration was identified at $K = 5$ and $u = 0{,}175$, which maximises Context Relevance in both datasets while keeping Groundedness between 73 % and 90 %.

A structural finding of the experiment is that above $u = 0{,}25$ the cosine distance filter no longer discards any candidates, because the empirical distribution of distances of the embedding model on the corpus never exceeds that value. This defines three regimes: a restrictive regime ($u = 0{,}1$) where almost all candidates are filtered out and the model abstains by default, a useful regime ($0{,}125 \leq u \leq 0{,}2$) where the filter actively shapes the context, and a saturated regime ($u \geq 0{,}25$) where the threshold has no effect. The comparison between test sets reveals an asymmetry that conditions the interpretation of the results: on questions with explicit bibliographic references, the proportion of high scores (≥ 4) falls to 36.7 % for Context Relevance, while it remains at 73.3 % for Groundedness, indicating that the bottleneck of the system is the retrieval phase rather than the generation phase.

For the SQL module, given the binary nature of correctness in this task, an iterative failure analysis was conducted over a representative hybrid query. The analysis revealed four layers of failure that emerge sequentially as each is mitigated: hallucination of non-existent columns when the schema is wide and encodes

variable and depth jointly, elision of long enumerations of columns through ellipsis when the context window is extended, accidental Cartesian products from joins on partial keys, and hallucination during the final synthesis phase when the DataFrame to be summarised exceeds the model's tabular processing capacity. The four layers, taken together, characterise the cognitive limits of an 8B-parameter model in this setting.

The work demonstrates that a hybrid SQL+RAG system can be successfully built and deployed entirely locally to answer natural-language questions about a complex environmental domain. Experimental results show that the intent classifier is highly accurate, the RAG module produces well-grounded answers (identifying retrieval rather than generation as the primary bottleneck), and the modular architecture proves highly effective for isolating component failures. The main limitations identified concern the retrieval quality of the embeddings model on bibliographic queries, the cognitive capacity of an 8B-parameter LLM to handle complex schemas and massive tabular datasets, the modest size of the test sets, and the reliance on a single LLM-as-judge for evaluation. Consequently, the most logical future directions include the introduction of hybrid dense-lexical retrieval strategies, the normalisation of database schemas, the evaluation of larger-scale language models to overcome current cognitive limits, and the incorporation of human annotation into the evaluation protocol. Ultimately, the system provides a functional baseline for querying both empirical data and scientific literature, offering a robust methodological framework upon which a more mature operational system can be developed.

Contenidos

1 Introducción **1**

2 Estado del arte y marco teórico **3**
2.1 Modelos de lenguaje y técnicas de prompting 3
2.2 Generación aumentada por recuperación (RAG) 4
2.3 Generación de SQL mediante LLMs (Text-to-SQL) 5
2.4 Sistemas híbridos 6

3 Análisis de objetivos y metodología **8**
3.1 Contexto y objetivos 8
3.2 Metodología de desarrollo 9
3.3 Descripción de los datos 10
3.3.1 Datos empíricos 10
3.3.2 Literatura científica 11

4 Diseño **12**
4.1 Visión general de la arquitectura 12
4.2 Elección de tecnologías 13
4.2.1 Base de datos analítica 13
4.2.2 Base de datos vectorial 14
4.2.3 Modelo de embeddings 15
4.2.4 Plataforma de inferencia local 15
4.2.5 Framework de orquestación 15
4.2.6 Entorno de ejecución y versiones 15
4.3 Preprocesamiento de los datos empíricos 17
4.4 Clasificador de intención 19
4.4.1 Enfoque basado en similitud semántica 19
4.4.2 Detección de empates y ruta híbrida 20
4.5 Módulo SQL 20
4.5.1 Diseño del prompt 20
4.5.2 Generación robusta y autocorrección 21
4.5.3 Capa de seguridad (*guardrails*) 22
4.5.4 Trazabilidad 23

4.6 Módulo RAG 23
4.6.1 Extracción y procesamiento 23
4.6.2 Recuperación de información 24
4.7 Orquestador 25
4.7.1 Diseño del prompt del modo RAG 26
4.7.2 Adaptación del prompt según la forma del resultado SQL . . 27
4.8 Detalles de implementación 28
4.8.1 Organización del código y entorno de ejecución 28
4.8.2 Convención de prefijos del modelo de embeddings 29

5 Experimentos y resultados **31**
5.1 Metodología de evaluación 31
5.2 Evaluación del clasificador de intención 32
5.2.1 Diseño experimental 32
5.2.2 Evaluación con empate 33
5.2.3 Evaluación sin empate 37
5.2.4 Impacto del tamaño del banco de ejemplos 41
5.3 Evaluación del módulo RAG 43
5.3.1 Diseño experimental 43
5.3.2 Régimen del umbral: descubrimiento de la saturación 44
5.3.3 Compromiso entre K y métricas 48
5.3.4 Comportamiento en el régimen útil y cuello de botella . . . 51
5.4 Análisis de fallos en el módulo de generación SQL 54
5.4.1 Primera capa: inferencia de columnas inexistentes 55
5.4.2 Segunda capa: desbordamiento de contexto y elisión léxica . 56
5.4.3 Tercera capa: producto cartesiano por JOIN parcial 57
5.4.4 Cuarta capa: alucinación en la etapa de síntesis 57
5.4.5 Valoración conjunta del módulo 58

6 Conclusiones y vías futuras **60**
6.1 Conclusiones principales 60
6.2 Limitaciones del trabajo 61
6.3 Vías futuras 62

Bibliografía **64**

Lista de Acrónimos y Abreviaturas **67**

Índice de figuras

4.1 Diagrama de la arquitectura del sistema 13

5.1 Evolución del parámetro k 34
5.2 Métricas globales 35
5.3 Matriz de confusión 36
5.4 F1-Score por clase 36
5.5 Evolución del parámetro K sin margen de empate 38
5.6 Métricas globales sin margen de empate 39
5.7 Matriz de confusión (sin empate) 40
5.8 F1-Score por clase (sin empate) 40
5.9 Métricas globales con banco reducido 42
5.10 Matriz de confusión (banco reducido) 42
5.11 F1-Score por clase (banco reducido) 42
5.12 Mapas de calor de groundedness por (K, u) para el dataset `AUTORES`. 46
5.13 Mapas de calor de groundedness por (K, u) para el dataset `SIMPLES`. 47
5.14 Distribución de notas en el régimen saturado ($u \geq 0{,}25$) partiendo del límite útil ($u = 0{,}175$) para el dataset `AUTORES`. 49
5.15 Distribución de notas en el régimen saturado ($u \geq 0{,}25$) partiendo del límite útil ($u = 0{,}175$) para el dataset `SIMPLES`. 50
5.16 Distribución de notas de context relevance en los regímenes restrictivo y útil ($u \in [0{,}1, 0{,}2]$) para el dataset `AUTORES`. 53
5.17 Distribución de notas de context relevance en los regímenes restrictivo y útil ($u \in [0{,}1, 0{,}2]$) para el dataset `SIMPLES`. 53

Índice de tablas

4.1 Especificación del entorno de ejecución, dependencias de software y modelos locales. 16

5.1 Precisión, recall y F1 por clase derivados de la matriz de confusión. 36
5.2 Precisión, recall y F1 por clase en ausencia del margen. 40
5.3 Precisión, recall y F1 por clase con el banco de ejemplos reducido. . 43
5.4 Regímenes empíricos del umbral de distancia coseno. 45
5.5 Resultados medios en el régimen saturado del umbral ($u = 0{,}25$). . 48
5.6 Resultados para $K = 5$ a lo largo del régimen útil del umbral. . . . 52

Listings

5.1 Primera capa de fallo. El modelo genera identificadores no validos en cada reintento sin converger en el esquema real de la base de datos. 55
5.2 Segunda capa de fallo. Fenómeno de elisión en listas largas de columnas, generando un fragmento de código incompatible con la sintaxis SQL. 56
5.3 Tercera capa de fallo. Datos cruzados por el JOIN parcial donde los valores de fecha se duplican de manera incorrecta para cada medición. 57
5.4 Cuarta capa de fallo. Fragmento de la salida del modulo de síntesis donde el modelo genera hechos falsos debido a la saturación de información tabular. 58

1 Introducción

La adopción de los *Large Language Models* (LLMs) en aplicaciones de procesamiento del lenguaje natural ha modificado en pocos años la forma en que los usuarios pueden interactuar con sistemas informáticos complejos. Lo que antes requería conocer un dialecto de consulta específico o navegar por interfaces estructuradas se puede expresar ahora mediante preguntas formuladas en lenguaje natural, sin que el usuario deba dominar los detalles técnicos de la fuente de información subyacente. Este cambio resulta especialmente relevante en dominios técnicos y científicos, donde la información útil suele estar dispersa en repositorios heterogéneos a los que solo pueden acceder usuarios especializados.

Sin embargo, los LLMs por sí solos no resuelven el problema de fondo, ya que su conocimiento queda fijado en el momento del entrenamiento y carecen de mecanismos para acceder de forma fiable a fuentes externas. Cuando se les interroga sobre datos concretos o sobre publicaciones específicas, tienden a generar respuestas plausibles pero infundadas, un fenómeno conocido como alucinación [7]. Para mitigar esta limitación se han consolidado dos enfoques complementarios pero distintos. Por un lado, la generación de consultas SQL sobre bases de datos estructuradas a partir de la pregunta del usuario, conocida como *Text-to-SQL* [15, 23]. Por otro lado, la *Retrieval-Augmented Generation* (RAG), que recupera fragmentos relevantes de un corpus documental para fundamentar la respuesta del modelo [5, 9].

En la práctica, ninguno de los dos enfoques es suficiente por sí mismo en muchos escenarios reales. Los datos cuantitativos no se interpretan sin el marco teórico que les da sentido, y la literatura científica difícilmente puede sustituir las mediciones empíricas cuando la pregunta exige una cifra concreta. Por tanto, surge la necesidad de sistemas híbridos capaces de combinar ambas fuentes y de decidir, para cada consulta, qué componentes deben intervenir y cómo se sintetiza el resultado final.

En este contexto, el presente Trabajo Fin de Grado plantea el diseño, la implementación y la evaluación de un sistema híbrido que integra los enfoques Text-to-SQL y RAG en un único pipeline. El sistema se estructura en cuatro componentes desacoplados —un clasificador de intención, un módulo SQL, un módulo RAG y un orquestador central— y se ha validado utilizando como caso de uso el ecosis-

tema del Mar Menor, un escenario que reúne series temporales de monitorización ambiental y un corpus de literatura científica especializada, y que por tanto resulta especialmente adecuado para someter a prueba un sistema de esta naturaleza.

El resto del documento se organiza como sigue. El capítulo 2 revisa el estado del arte en torno a los enfoques Text-to-SQL, RAG y sus combinaciones. El capítulo 3 expone el contexto y los objetivos del trabajo, la metodología seguida durante el desarrollo y la descripción de los datos empleados. El capítulo 4 detalla las decisiones de diseño, las tecnologías y la implementación de cada uno de los componentes del sistema. El capítulo 5 presenta la evaluación experimental del clasificador de intención y del módulo RAG, así como un análisis de fallos del módulo SQL. Finalmente, el capítulo 6 recoge las conclusiones principales, las limitaciones del trabajo y las vías futuras de investigación.

2 Estado del arte y marco teórico

El sistema desarrollado en este TFG se sitúa en la intersección de tres líneas de investigación activas en el procesamiento del lenguaje natural: LLMs y las técnicas de prompting, la generación aumentada por recuperación, y la generación de SQL a partir de lenguaje natural. Este capítulo revisa el estado actual de estas tres áreas y, en su apartado final, contextualiza el presente trabajo frente a las propuestas recientes que integran recuperación documental y generación de SQL.

2.1. Modelos de lenguaje y técnicas de prompting

La arquitectura *Transformer* introducida por Vaswani et al. [19] es el sustrato común sobre el que se construyen los modelos de lenguaje contemporáneos. Su capacidad para modelar dependencias a larga distancia y para escalar a miles de millones de parámetros ha dado lugar a la familia de los LLMs, entre los que destacan GPT-3 [1], Llama 2 [17] y, más recientemente, Llama 3 [11], el modelo concreto utilizado en este trabajo. Una propiedad emergente de estos modelos a partir de cierta escala es el aprendizaje en contexto (*in-context learning*), es decir, la capacidad de aprender una tarea a partir de un puñado de ejemplos incluidos en el prompt, sin reentrenamiento del modelo.

Sobre esta capacidad se han construido dos técnicas que el presente trabajo utiliza de forma explícita. El *few-shot prompting* [1] consiste en incluir en el prompt ejemplos representativos del comportamiento esperado, con el doble objetivo de transmitir tanto el formato como el estilo de razonamiento. La cadena de pensamiento (*chain-of-thought prompting*), propuesta por Wei et al. [22], consiste en pedir al modelo que verbalice los pasos intermedios antes de emitir la respuesta final; en tareas que requieren razonamiento multipaso, esta verbalización mejora sustancialmente la precisión sin necesidad de modificar el modelo subyacente. Ambas técnicas se aplican en el módulo SQL descrito en la sección 4.5, donde la cadena de pensamiento se materializa en un bloque `<thought_process>` previo a la emisión del código SQL.

A pesar de estos avances, los LLMs presentan una limitación bien documentada, como es la alucinación [7], entendida como la generación de afirmaciones plausibles

pero infundadas. Esta limitación es especialmente crítica en dominios técnicos o científicos, donde una respuesta incorrecta puede pasar desapercibida si no se contrasta con una fuente externa. Las arquitecturas RAG, comentadas en el siguiente apartado, son el principal mecanismo propuesto en la literatura para mitigar este problema.

2.2. Generación aumentada por recuperación (RAG)

El paradigma RAG fue introducido por Lewis et al. [9] como una forma de combinar la capacidad generativa de los LLMs con la precisión de los sistemas de recuperación de información. Una arquitectura RAG clásica consta de tres componentes: un codificador que proyecta documentos y consultas a un espacio vectorial común, un índice vectorial que permite recuperar los fragmentos más similares a la consulta, y un generador que produce la respuesta condicionado por dichos fragmentos. Los embeddings densos para la recuperación, en sustitución de los modelos clásicos basados en Best Match 25 (BM25), se popularizaron a partir del trabajo sobre *Dense Passage Retrieval* de Karpukhin et al. [8].

En los años posteriores, el ecosistema RAG ha evolucionado en varias direcciones que el survey de Gao et al. [5] sistematiza en tres paradigmas: el RAG naïf, en el que recuperación y generación operan de forma desacoplada; el RAG avanzado, que introduce reordenamiento, filtrado y reescritura de consultas; y el RAG modular, que descompone el pipeline en componentes intercambiables. El sistema del presente TFG se inscribe en el paradigma modular, en tanto que el módulo RAG se integra con un módulo SQL y un clasificador de intención bajo un orquestador común.

Una contribución específica relevante para este trabajo es la familia de modelos de embeddings EmbEddings from bidirEctional Encoder rEpresentations (E5) [20], entrenada mediante aprendizaje contrastivo débilmente supervisado sobre pares consulta-documento, y su variante multilingüe [21]. Esta familia condiciona la representación generada en función del rol del texto mediante un prefijo de instrucción (`query:` para preguntas, `passage:` para documentos), una asimetría que se aprovecha en el módulo RAG y que se discute en detalle en la sección 4.8.2.

La evaluación de sistemas RAG plantea un reto, ya que generalmente una consulta no tiene una única respuesta correcta. Una práctica presente en la literatura es la evaluación mediante un modelo juez (*LLM-as-judge*), formalizada por Zheng

et al. [24]. Esta metodología consiste en delegar la valoración de las respuestas en otro LLM, instruido mediante un prompt que define las dimensiones a evaluar. Las dos métricas más utilizadas en la evaluación de sistemas RAG son la *Context Relevance*, que mide la utilidad de los fragmentos recuperados para responder a la pregunta, y la *Groundedness*, que mide la fidelidad de la respuesta respecto a la evidencia recuperada [4, 10]. Esta es la metodología que se sigue en la evaluación del módulo RAG presentada en el capítulo 5.

2.3. Generación de SQL mediante LLMs (Text-to-SQL)

La traducción automática de lenguaje natural a consultas SQL es un problema con tradición previa al auge de los LLMs, pero su rendimiento dio un salto cualitativo con la aparición de estos. El benchmark de referencia en este ámbito es Spider [23], un conjunto de datos muy diverso que se ha consolidado como el banco de pruebas estándar de la comunidad.

Entre los enfoques basados en LLMs, el trabajo de Pourreza y Rafiei [13] introdujo DIN-SQL, una propuesta que descompone la tarea en subtareas (vinculación de esquema, clasificación de complejidad, generación y autocorrección) y que combina prompting few-shot con cadenas de pensamiento. Esta estrategia de descomposición y autocorrección es la que inspira el bucle de cinco reintentos del módulo SQL descrito en la sección 4.5.2, de modo que la respuesta del modelo no se acepta a ciegas, sino que se valida y, en caso de error, se reinyecta la información del fallo al propio modelo para que la corrija de forma autónoma.

El survey de Shi et al. [15] ofrece una panorámica de los métodos de Text-to-SQL basados en LLMs, organizándolos en torno a tres ejes, que abarcan el formato de representación del esquema, las técnicas de prompting empleadas y los mecanismos de validación o autocorrección. El sistema del presente TFG se alinea con las recomendaciones del survey en los tres ejes: inyecta el esquema real para evitar suposiciones, combina cadena de pensamiento y few-shot prompting, e incorpora un bucle de autocorrección guiado por los mensajes de error del propio motor de base de datos.

A diferencia de los bancos de pruebas estándar como Spider, que evalúan a los modelos sobre bases de datos transaccionales (Procesamiento de Transacciones en Línea (OLTP)) con esquemas acotados y vocabularios de propósito general, el

escenario de este trabajo presenta dos retos técnicos distintos. El primero es la naturaleza analítica de las consultas, como agregaciones sobre series temporales o filtrados por estación y fecha, que requieren el uso de DuckDB [14], un motor de Procesamiento Analítico en Línea (OLAP) integrado cuya elección se justifica en la sección 4.2.1. El segundo reto es la heterogeneidad del propio esquema. Al tratarse de un entorno real, el modelo debe enfrentarse a nomenclaturas técnicas que escapan a su distribución de entrenamiento habitual, llegando a presentar tablas cuyos nombres codifican simultáneamente la variable física y el parámetro de medición. Esta disonancia entre los esquemas limpios con los que se evalúan los LLMs y la complejidad del dominio real incrementa la tasa de error sintáctico y lógico, lo que convierte al bucle de autocorrección en un mecanismo de seguridad necesario para la viabilidad de la arquitectura.

2.4. Sistemas híbridos

La idea de combinar generación de SQL y recuperación documental en un único sistema ha cobrado gran relevancia recientemente bajo el paraguas del paradigma *Agentic RAG* [16], que extiende el RAG clásico introduciendo agentes capaces de invocar herramientas externas en función de la consulta. En este marco, Text-to-SQL deja de ser un sistema independiente para convertirse en una herramienta más a la que un agente orquestador puede recurrir cuando la pregunta requiere datos estructurados, mientras que el RAG se reserva para las consultas que dependen de evidencia documental.

Algunos trabajos recientes han explorado esta combinación de forma explícita. Aunque la literatura sobre sistemas híbridos SQL+RAG es todavía escasa y se encuentra en gran medida fuera de los benchmarks académicos clásicos, los marcos de desarrollo como LangChain [2] han facilitado la integración de ambos paradigmas en aplicaciones reales, lo que ha dado lugar a un número creciente de prototipos y casos de uso documentados en la literatura gris y en publicaciones de divulgación técnica.

El sistema desarrollado en este TFG se sitúa en esta línea, con tres particularidades que lo distinguen de los desarrollos más habituales. En primer lugar, el clasificador de intención no se delega en un LLM, sino que se implementa mediante similitud semántica sobre embeddings, lo que reduce el coste computacional por consulta sin sacrificar la calidad del enrutamiento. En segundo lugar, el módulo SQL incorpora una capa de seguridad explícita basada en análisis léxico de la consulta generada, una salvaguarda que no suele aparecer en los prototipos basados en

frameworks de orquestación de propósito general. Por último, todo el sistema opera de forma local, sin dependencia de Interfaz de Programación de Aplicacioness (APIs) externas, lo que lo distingue de la mayoría de las propuestas de la literatura, que recurren a modelos comerciales de gran tamaño accesibles únicamente vía API.

En definitiva, el trabajo se apoya en los pilares metodológicos descritos en este capítulo (arquitectura SQL modular, prompting basado en cadenas de pensamiento y few-shot, autocorrección guiada por feedback de ejecución, y evaluación mediante modelo juez) y los integra en un sistema cuya contribución reside en la combinación cuidadosa de las técnicas y en su evaluación empírica detallada en un escenario realista, antes que en la introducción de técnicas nuevas.

3 Análisis de objetivos y metodología

3.1. Contexto y objetivos

En la introducción (capítulo 1) hemos planteado la necesidad de un sistema híbrido Text-to-SQL/RAG y se ha esbozado su arquitectura en cuatro componentes. Esta sección recoge esa propuesta y la convierte en un conjunto de decisiones, detallando qué exige construir un sistema de este tipo, cómo se concretan los objetivos del trabajo y sobre qué caso de uso se valida.

La construcción de un sistema híbrido plantea dos exigencias que condicionan el resto del capítulo. La primera es el enrutamiento de cada consulta hacia el componente adecuado, ya que el sistema debe decidir de forma autónoma si una pregunta requiere acceso a los datos empíricos, a la literatura científica o a una combinación de ambas fuentes. La segunda es la coordinación entre componentes, dado que los resultados que produce cada módulo deben integrarse en una única respuesta final sin que la información pierda fidelidad en el proceso, lo que justifica que la generación de la respuesta se centralice en un orquestador en lugar de delegarse en cada módulo por separado.

A partir de estas exigencias, el objetivo general de este Trabajo de Fin de Grado se concreta en cuatro objetivos específicos. En primer lugar, diseñar un clasificador de intención eficiente que decida de forma autónoma la ruta que debe seguir cada consulta, evitando recurrir a un LLM para una tarea de enrutamiento. En segundo lugar, implementar un módulo SQL capaz de generar consultas correctas sobre el esquema real de la base de datos y de recuperarse de los errores que se produzcan durante la generación. En tercer lugar, implementar un módulo RAG que recupere fragmentos relevantes de un corpus documental y que minimice la alucinación del modelo generador mediante un prompt orientado a la fundamentación en la evidencia. Por último, integrar los tres componentes anteriores mediante un orquestador que centralice la generación de la respuesta final, garantizando que tanto los datos como la literatura lleguen al modelo sin reinterpretaciones intermedias que comprometan su fidelidad.

La validación de esta arquitectura se ha llevado a cabo sobre el caso de uso del Mar Menor y su cuenca hidrográfica. Se trata de un ecosistema afectado por una grave crisis ecológica de contaminación y eutrofización —debida principalmente al impacto de la agricultura intensiva y a la actividad minera histórica de la zona— , lo que ha motivado el despliegue de varias redes de sensores y la publicación de un volumen considerable de literatura científica especializada. La coexistencia de ambos cuerpos de información en un mismo dominio convierte al Mar Menor en un escenario especialmente adecuado para someter a prueba un sistema de las características aquí planteadas, ya que reúne las dos naturalezas de información que motivan la arquitectura híbrida. La descripción detallada de los datos empíricos y del corpus documental utilizados se aborda en la sección 3.3.

3.2. Metodología de desarrollo

El diseño e implementación del sistema se basó en una metodología de desarrollo iterativa e incremental. En lugar de trabajar con la totalidad del conjunto de datos desde el inicio, se optó por la construcción de un producto mínimo viable, lo que permitió acotar los posibles errores y validar de forma más controlada la calidad de cada componente antes de escalar el sistema.

En una primera fase, el desarrollo se centró en la construcción de los módulos SQL y RAG. En ambos se buscó que la calidad de las respuestas a las consultas del usuario fuese satisfactoria, utilizando únicamente un Comma Separated Values (CSV) y un documento respectivamente. Una vez que se consideró que el rendimiento individual era el adecuado, se diseñó el `orquestador` para integrar ambos flujos en un único pipeline.

La construcción del orquestador supuso la creación de un enrutador, es decir, el clasificador de intención. Para su implementación se tuvo como objetivo diseñar un componente eficiente —tanto en relación con el coste computacional como en rendimiento— que fuese capaz de derivar la consulta del usuario al módulo correspondiente de forma completamente autónoma.

A partir de esta primera versión funcional e integrada del sistema, se llevó a cabo una fase de optimización y ajuste de configuración (*tuning*). Esta etapa incluyó una exhaustiva batería de pruebas orientadas a determinar los hiperparámetros óptimos para cada componente.

Tras asegurar que la arquitectura base ofrecía un rendimiento robusto y de alta

calidad, se dio paso a escalar el conjunto de datos con el que se estaba trabajando. La mayor dificultad fue realizar un preprocesamiento de los datos de forma que estuviesen en las mejores condiciones para ser utilizados.

Finalmente, como culminación del proceso de desarrollo, se ejecutaron pruebas de validación sobre la arquitectura completa.

3.3. Descripción de los datos

3.3.1. Datos empíricos

Los datos empíricos utilizados en este trabajo provienen de la monitorización del ecosistema del Mar Menor y su cuenca hidrográfica en el marco del proyecto OMM-Azul del Instituto Murciano de Investigación y Desarrollo Agrario y Medioambiental (IMIDA). Esta serie temporal abarca aproximadamente desde 2017 hasta 2025, con variaciones según la fuente de origen.

La red meteorológica de la Agencia Estatal de Meteorología (AEMET) proporciona datos climáticos de temperatura, precipitación y humedad registrados en distintas estaciones terrestres de la zona. Por otra parte, los datos procedentes de las boyas están divididos en dos CSV. El primero corresponde a las boyas `NAUTILUS`, diseñadas y construidas por el grupo de investigación `PRISMA` [18] de la Universidad Politécnica de Cartagena (UPCT) y operadas por el IMIDA, que recogen mediciones de oxígeno disuelto, conductividad y temperatura con una periodicidad de un minuto. Dado que en estas boyas los registros de profundidad presentan valores variables, se ha considerado mantener sus mediciones en un fichero independiente. El segundo corresponde a una unión entre las boyas de la UPCT y las del IMIDA, que comparten estaciones y localizaciones geográficas y miden variables como temperatura, salinidad, oxígeno disuelto, clorofila, turbidez, conductividad, pH, materia orgánica y ficoeritrina a profundidades fijas de entre -0.5m y -6m, lo que ha permitido unificarlas en un único fichero al compartir un esquema compatible. Por último, los piezómetros del Sistemas Automáticos de Información Hidrológica (SAIH) registran, a través de 20 puntos de sondeo distribuidos por la cuenca hidrográfica, variables como temperatura, conductividad, nivel piezométrico, salinidad y total de sólidos disueltos. Los aforadores de ramblas del mismo sistema miden variables meteorológicas e hidrológicas como temperatura, precipitación, caudal, nivel del agua, humedad, dirección y velocidad del viento y presión atmosférica, aunque el conjunto de variables presentes en los datos varía según el sondeo o sensor.

El conjunto de datos más voluminoso supera los 9 millones de registros, lo que motiva el uso de DuckDB como motor de consulta analítica, cuya elección se detalla en la sección 4.2.1.

3.3.2. Literatura científica

El corpus utilizado para el módulo RAG está compuesto por 36 documentos científico-técnicos en formato PDF, procedentes de bases de datos académicas y repositorios institucionales como el Instituto Español de Oceanografía (IEO-CSIC) y la Comunidad Autónoma de la Región de Murcia (CARM). La literatura cubre tres ámbitos principales: el diagnóstico ecológico del Mar Menor y su cuenca, la aplicación de teledetección mediante satélites para la estimación de variables de calidad del agua, y el uso de técnicas de *machine learning* e inteligencia artificial para la modelización hidrológica y la predicción de variables biofísicas.

Dado el alcance de la investigación, la colección integra publicaciones tanto en español como en inglés. Si bien algunos de estos textos analizan otros ecosistemas acuáticos a nivel internacional, su inclusión cumple una función metodológica y comparativa, manteniéndose el Mar Menor como el eje central de todo el corpus. Esta base de conocimiento teórico es fundamental para dotar al sistema del contexto científico necesario, permitiendo al orquestador interpretar los datos empíricos basándose en literatura contrastada.

4 Diseño

En este capítulo se describe la arquitectura del sistema desarrollado. Para ello, se comienza con una visión global del pipeline y se profundiza a continuación en cada uno de sus componentes: el clasificador de intención, el módulo SQL, el módulo RAG y el orquestador. Para cada uno de ellos se explican las decisiones de diseño tomadas, así como su rol dentro del sistema.

4.1. Visión general de la arquitectura

Como se ilustra en la Figura 4.1, el sistema se organiza como un pipeline de cuatro etapas que clasifica una pregunta en lenguaje natural para luego proporcionar una respuesta fundamentada en datos empíricos y/o literatura científica. Además, también se puede observar que se ha diseñado con un enfoque modular y desacoplado, con el objetivo de garantizar la independencia funcional de sus componentes. Dicha independencia permite que el sistema sea escalable, flexible y que la actualización o sustitución de las partes individuales no altere el resto del pipeline.

El flujo de ejecución del sistema se puede resumir en las siguientes cuatro fases:

1. **Recepción de la consulta:** El usuario hace una consulta en lenguaje natural a través de la terminal, la cual es recibida por el orquestador y delegada al clasificador de intención.

2. **Clasificación de la consulta:** La clase `ClasificadorIntencion` determina si para resolver la consulta es necesaria la base de datos empíricos (SQL), recuperar la información de la literatura científica (RAG) o una síntesis de ambas fuentes (`HIBRIDO`)

3. **Ejecución del módulo correspondiente:** Según el camino seguido, interviene el módulo SQL, el módulo RAG o ambos de forma secuencial.

4. **Generación de la respuesta final:** Finalmente, el orquestador recibe los datos producidos en el paso anterior y utiliza un LLM para generar una respuesta coherente en lenguaje natural.

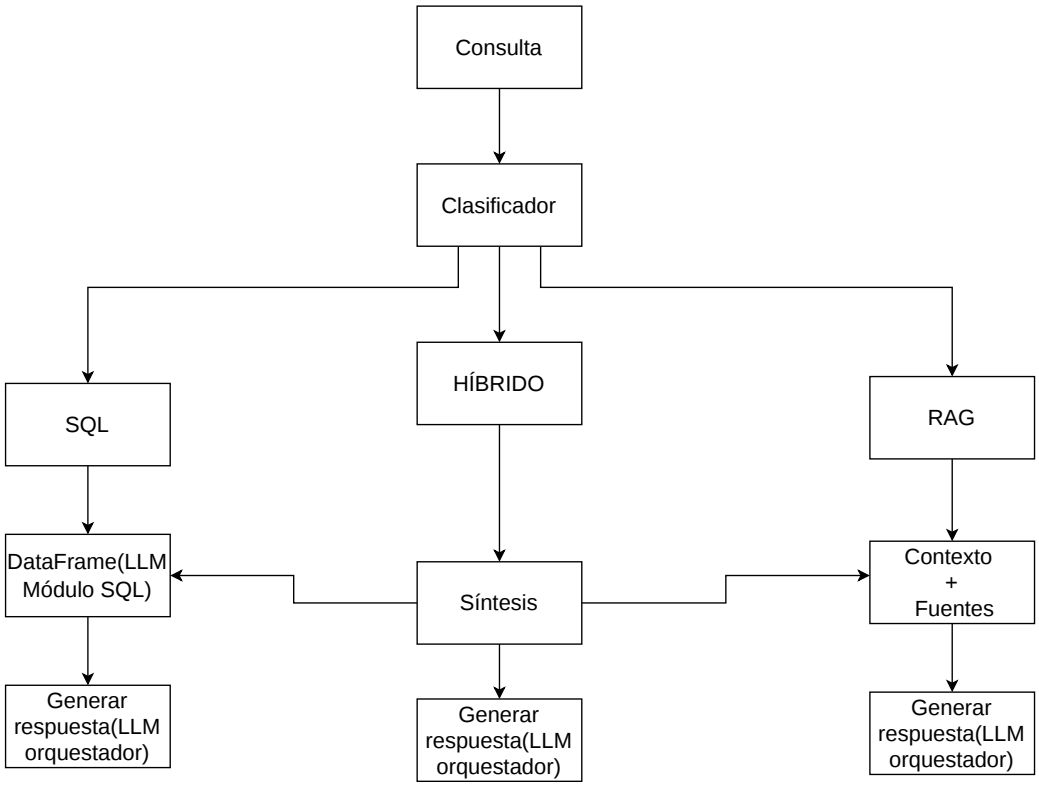


Figura 4.1: Diagrama de la arquitectura del sistema

4.2. Elección de tecnologías

Las decisiones tecnológicas del sistema se tomaron atendiendo a criterios de rendimiento, coste computacional, privacidad de los datos y adecuación al caso de uso.

A continuación se justifica cada una de las decisiones principales.

4.2.1. Base de datos analítica

Para la gestión y el análisis de los datos empíricos, se ha seleccionado `DuckDB` como motor de base de datos, ya que a diferencia de los Sistema de Gestión de Bases de Datos Relacionales (RDBMS) tradicionales como `PostgreSQL` o `MySQL` que son orientados al OLTP, DuckDB está diseñado específicamente para OLAP.

Esta elección se fundamenta en tres características clave para el procesamiento de las series temporales requeridas en las consultas del usuario.

- **Arquitectura orientada a columnas:** A diferencia de las bases de datos orientadas a filas (como SQLite), DuckDB almacena los datos por columnas. Esto permite ejecutar agregaciones y consultas sobre grandes volúmenes de datos de forma eficiente, minimizando las operaciones de entrada/salida (I/O) en disco [3].
- **Ejecución en proceso (*in-process*):** DuckDB opera sin necesidad de desplegar, configurar o mantener un servidor de base de datos dedicado. Se ejecuta directamente dentro del proceso del script de Python, eliminando por completo la latencia de red en la transferencia de datos entre el cliente y el servidor [3].
- **Integración y procesamiento vectorizado:** El motor está optimizado para el ecosistema de ciencia de datos de Python, permitiendo ejecutar consultas SQL directamente sobre DataFrames de Pandas y ficheros CSV sin requerir pasos previos de importación. Además, su motor de ejecución vectorizada procesa los datos en lotes (*batches*), aprovechando la caché de las Unidad Central de Procesamiento (CPU) modernas [3].

Estas características aseguran que el módulo SQL pueda ejecutar operaciones analíticas complejas generadas por el LLM de forma eficiente y sin requerir infraestructura adicional.

4.2.2. Base de datos vectorial

Para la persistencia y recuperación de los embeddings, se ha seleccionado ChromaDB como base de datos. Esta elección se debe, principalmente, a que ofrece una integración directa y eficiente para operar con espacios vectoriales. Además, existen factores técnicos que refuerzan esta decisión, destacando su capacidad de ejecución local. Al trabajar en un entorno local, se evita la transferencia de datos a la nube, garantizando una mayor privacidad de la información y reduciendo la latencia de las operaciones.

Por otra parte, ChromaDB gestiona la persistencia de los datos de forma automática en el sistema de ficheros y permite realizar filtrados basados en metadatos (como el nombre del archivo de origen o la página del PDF), una característica esencial para garantizar la trazabilidad en el módulo RAG.

Por último, es considerado el estándar de facto para el desarrollo de prototipos en LangChain. Su sencilla configuración permite desplegar rápidamente un sistema RAG funcional, lo que facilita centrar los esfuerzos en la lógica del modelo y la optimización de los resultados.

4.2.3. Modelo de embeddings

Para la vectorización de documentos y consultas se ha seleccionado el modelo `intfloat/multilingual-e5-small` [21], disponible en `Hugging Face`. Su elección se justifica por tres razones principales. En primer lugar, ofrece soporte multilingüe, lo que resulta relevante dado que tanto los documentos científicos como las consultas del usuario pueden estar en distintos idiomas. En segundo lugar, se ejecuta completamente en local sobre CPU, sin necesidad de Unidad de Procesamiento Gráfico (GPU), ni de llamadas a servicios externos, lo que refuerza la privacidad del sistema. Por último, a diferencia de un LLM, no requiere inferencia generativa completa, lo que lo hace considerablemente más eficiente en términos de coste computacional para la tarea de recuperación semántica.

4.2.4. Plataforma de inferencia local

Para la ejecución de los LLM se ha utilizado `Ollama` [12], una plataforma que permite desplegar y ejecutar LLM de forma local sin dependencia de servicios externos. Concretamente, se ha utilizado el modelo `Llama 3`, ejecutado íntegramente en la máquina local. Esta decisión elimina la necesidad de transmitir datos a APIs remotas, garantizando la privacidad de las consultas y los datos del sistema, y suprime la latencia y los costes asociados al uso de servicios en la nube.

4.2.5. Framework de orquestación

LangChain fue empleado como framework de orquestación, proporcionando abstracciones de alto nivel para la integración entre los distintos componentes del sistema: el modelo de embeddings, la base de datos vectorial y el LLM. Su uso permitió reducir el código necesario para conectar estos elementos y facilitó la iteración durante el desarrollo al ofrecer interfaces unificadas para operaciones como la carga de documentos, el *chunking* y la consulta a la base de datos vectorial.

4.2.6. Entorno de ejecución y versiones

Las decisiones tecnológicas justificadas en los apartados anteriores se concretaron en un entorno físico y lógico específico. La evaluación del sistema y el despliegue de los modelos locales se ejecutaron sobre un equipo portátil con las siguientes características de *hardware*:

- **Procesador (CPU):** Intel Core i7-12700H.
- **Memoria RAM:** 32 GB.

- **Tarjeta Gráfica (GPU):** NVIDIA GeForce RTX 3060 (edición para portátiles).
- **Sistema Operativo:** Entorno de ejecución basado en el Subsistema de Windows para Linux (WSL).

Sobre esta infraestructura, se fijaron de forma explícita las versiones de las dependencias de *software* para garantizar la reproducibilidad de los resultados presentados en el capítulo de evaluación. La Tabla 4.1 detalla el entorno completo de librerías y modelos sobre el cual se desarrolló, probó y ejecutó la arquitectura integrada.

Tabla 4.1: Especificación del entorno de ejecución, dependencias de software y modelos locales.

Componente	**Versión**
Python	`3.12`
DuckDB	`1.4.4`
ChromaDB	`1.4.1`
LangChain	`1.2.9`
`langchain-community`	`0.4.1`
`langchain-huggingface`	`1.2.0`
`langchain-chroma`	`1.1.0`
`langchain-ollama`	`1.0.1`
Ollama (Motor de inferencia)	`0.15.4`
Modelo de Lenguaje (LLM)	`Llama 3 (8B)` — Etiqueta: `llama3:latest` Digest criptográfico: `365c0bd3c000` (4.7 GB)
Modelo de *embeddings*	`intfloat/multilingual-e5-small`

Es preciso detallar dos aspectos orientados a la reproducibilidad del sistema. En primer lugar, dado que la invocación del modelo de lenguaje desde el orquestador se realiza mediante el identificador genérico `llama3`, una etiqueta que resulta ambigua debido a las actualizaciones periódicas del registro de Ollama, se hace constar formalmente el *digest* SHA de la variante de 8B parámetros empleada. Este identificador actúa como una firma inmutable del estado del modelo en la máquina anfitriona.

En segundo lugar, el modelo de embeddings queda vinculado a su ruta del repositorio de `Hugging Face`, lo que asegura que el espacio vectorial y las métricas de distancia aplicadas mantengan una consistencia matemática a lo largo de todas las evaluaciones del sistema.

4.3. Preprocesamiento de los datos empíricos

Los datos en bruto, procedentes de las cinco fuentes previamente descritas, se encontraban distribuidos en 316 ficheros CSV individuales. Estos archivos presentaban formatos, separadores y estructuras internas heterogéneas, por lo que para agruparlos en cinco ficheros, se desarrolló un proceso completo de extracción, transformación y carga (ETL) implementado en Python mediante scripts independientes por fuente.

Extracción

La fase de extracción debió abordar las particularidades de cada origen de datos. Los ficheros de la AEMET incluían exactamente cuatro líneas de metadatos antes de la cabecera, por lo que podían omitirse directamente mediante el parámetro `skiprows`. Sin embargo, en el resto de fuentes el número de líneas de metadatos era variable, por lo que se implementó un mecanismo de detección que recorre cada fichero línea a línea hasta localizar aquella que comienza por `Date`, identificándola como la cabecera real.

Por otro lado, los ficheros de las boyas de la UPCT utilizaban de forma inconsistente el punto y coma (`;`) o el tabulador (`\t`) como separador de campos. Para solventarlo, la detección del separador se integró en el mismo recorrido de búsqueda de cabecera: una vez localizada la línea que comienza por `Date`, se comprueba si contiene un tabulador, y en ese caso se sustituye el separador por defecto.

Por último, de cada directorio de origen se excluyeron los ficheros de metadatos mediante filtrado por nombre de archivo.

Transformación y normalización

Una vez extraídos los datos, se aplicaron distintas transformaciones según la fuente. En todas ellas se añadió una columna identificadora denominada `Estacion`, `Boya` o `Sondeo` según el caso, que permite al sistema distinguir el origen geográfico de cada registro.

Los ficheros de la AEMET y de las boyas `NAUTILUS` presentaban un esquema más sencillo: cada fichero contenía ya todas las variables de una estación o boya en columnas separadas, por lo que no fue necesaria ninguna integración entre ficheros más allá de la concatenación vertical de todas las estaciones.

Sin embargo, para las boyas del IMIDA y de la UPCT, donde los datos de cada estación estaban distribuidos en un fichero por variable, fue necesario renombrar las columnas antes de integrarlas por dos motivos. El primero es que el nombre de la variable solo era identificable a partir del nombre del fichero y no de su contenido, por lo que al unir varios ficheros las columnas serían indistinguibles entre sí. El segundo es que ciertos ficheros de algunas estaciones no incluían las unidades de medida en los nombres de columna, lo que generaba inconsistencias. Para resolver ambos problemas, se construyeron nombres combinando el identificador de columna, el nombre de la variable y su unidad mediante un diccionario predefinido, obteniendo columnas como `Depth_-1m_Temperature_(°C)`. Posteriormente, los ficheros de cada estación se integraron mediante un *outer join* sobre la columna `Date`, de forma que cada fila agrupa todas las variables disponibles para un instante temporal dado.

Para los piezómetros y los aforadores de ramblas del SAIH, que también presentaban variables en ficheros separados, se aplicó el mismo esquema de integración, utilizando `Date` y `Time` conjuntamente como clave de unión.

En todos los casos, cuando una variable no tiene medición en un instante concreto tras haber hecho la unión externa, la celda correspondiente queda como valor nulo, preservando la integridad temporal de la serie.

Carga

Una vez transformados los datos de cada estación de forma individual, se procedió a su unión en ficheros finales mediante concatenación vertical. Para cada fuente, los DataFrames de todas las estaciones o sondeos procesados se apilaron en un único fichero CSV, obteniendo así un fichero por origen de datos.

Cabe recalcar que las boyas del IMIDA y de la UPCT requirieron un paso adicional: tras generar sus respectivos ficheros unificados de forma independiente, se realizó una segunda concatenación vertical entre ambos, añadiendo una columna `Lugar` con los valores `IMIDA` y `UPCT` respectivamente para preservar la trazabilidad del origen de cada medición. Dado que ambas redes no comparten exactamente el mismo conjunto de variables — la UPCT mide adicionalmente ficoeritrina, ausente en los datos del IMIDA — la concatenación introduce valores nulos en las columnas no compartidas, siguiendo el mismo criterio aplicado durante la transformación.

El resultado final del proceso es un total de 5 ficheros CSV, derivados de los aproximadamente 316 ficheros originales.

4.4. Clasificador de intención

El clasificador de intención es el primer módulo que entra en acción y el responsable de decidir el camino que seguirá el pipeline, es decir, clasifica la consulta en función de la fuente de datos necesaria para satisfacerla.

Su diseño se basa en similitud semántica utilizando un modelo de embeddings, ya que, a diferencia de un LLM, no requiere inferencia generativa completa, lo que reduce significativamente el coste computacional por consulta y resulta suficiente para los requisitos del sistema. Las expresiones regulares fueron descartadas como alternativa por su fragilidad ante la variabilidad natural del lenguaje del usuario.

4.4.1. Enfoque basado en similitud semántica

El clasificador contiene un conjunto de ejemplos para cada una de las categorías posibles (`SQL`, `RAG`, `HÍBRIDO`), que son vectorizados y almacenados en memoria. Luego, cada consulta que se recibe, se vectoriza y se calcula su similitud coseno con respecto a todos los ejemplos almacenados.

El proceso para determinar la categoría ganadora se basa en el algoritmo de *K-NN*. No obstante, en lugar de seleccionar los k vecinos más cercanos de forma global y decidir por mayoría de "votos", se ha optado por seleccionar los k ejemplos más similares dentro de cada categoría y promediar sus puntuaciones. Es decir, la categoría ganadora es aquella cuya media de similitud coseno en su *top-k* es mayor.

Esta decisión de diseño hace que el clasificador sea más robusto que si se utilizara un algoritmo K-NN, ya que se le da más importancia al valor real de la similitud que a la cantidad de "votos" conseguidos. Una categoría podría resultar ganadora simplemente por acumular más ejemplos cercanos a la consulta, aunque su similitud coseno fuese menor. Al promediar las mejores puntuaciones de cada clase, se consigue que la decisión final dependa de cuánto se parece semánticamente la pregunta a los ejemplos. La justificación empírica de este método y la elección del parámetro k se detallan en la sección 5.2.

Cabe señalar que, dado que el modelo de embeddings devuelve vectores normalizados, el cálculo de la similitud coseno se reduce al producto escalar entre vectores, lo que evita la normalización explícita en cada comparación y reduce el coste por consulta.

4.4.2. Detección de empates y ruta híbrida

Un aspecto crítico del diseño es el manejo de preguntas ambiguas que podrían pertenecer tanto a la categoría `SQL` como a la `RAG`. Para evitar que el sistema elija incorrectamente en estos casos, se establece un margen δ a partir del cual, si la diferencia de puntuación entre ambas categorías es inferior a dicho margen, el sistema clasifica automáticamente la consulta como `HIBRIDO`.

Es importante señalar que esta comprobación de empate se aplica únicamente cuando la categoría ganadora por promedio no es ya `HIBRIDO`. Esto quiere decir que si el clasificador ha decidido por mayoría de similitud que la consulta es híbrida, esa decisión prevalece sobre cualquier resolución posterior por margen.

4.5. Módulo SQL

El módulo SQL tiene la responsabilidad de transformar las consultas en lenguaje natural en consultas ejecutables sobre DuckDB, procesando los datos de los distintos CSV para devolver los resultados en forma de DataFrames. Para conseguirlo, se hace uso de un LLM que se ha diseñado usando un prompt que incorpora distintas técnicas en pos de conseguir un rendimiento óptimo dentro de las posibilidades.

4.5.1. Diseño del prompt

El rendimiento de este módulo está muy condicionado por el prompt, por lo que a continuación se detallan las decisiones más relevantes.

Ingeniería de contexto

Se inyecta el esquema real de la base de datos, garantizando que el modelo no opere sobre suposiciones y mitigando la alucinación de columnas o tipos de datos inexistentes.

Asignación de rol

Para garantizar la calidad en la generación, se implementa una técnica de asignación de rol (*role prompting*). Mediante ella, se busca que el modelo asuma un rol de alta fidelidad, es decir, se define al agente como un "ingeniero de datos especializado en DuckDB y SQL OLAP de alto rendimiento", con rasgos de comportamiento específicos: eficiencia, precisión técnica y un sesgo marcado hacia la verificación antes de la implementación. Asumir dicho rol implica dotar al modelo de un conocimiento profundo de las cargas de trabajo OLAP, de la eficiencia del

almacenamiento en columnas y de los matices sintácticos específicos del dialecto de DuckDB. Para completar este perfil, se instruye al agente para que adopte una postura crítica ante los datos, cuestionando siempre la estructura del esquema antes de realizar consultas.

Cadena de pensamiento

Para garantizar la fiabilidad del código generado, el diseño incorpora un protocolo de `cadena de pensamiento` (*chain-of-thought* [22]) a través de un bloque denominado `<thought_process>`. Con esto se consigue que el agente planifique la consulta antes de emitir el bloque de código SQL final. Esta separación entre el razonamiento y la ejecución permite al modelo identificar errores lógicos antes de emitir la consulta a la base de datos, y constituye además el punto en el que se reinyecta la información de los fallos durante la autocorrección, descrita en la sección 4.5.2.

Few-shot prompting

El prompt incluye un conjunto de ejemplos representativos siguiendo la técnica de *few-shot prompting* [1], con el objetivo de guiar al modelo en el estilo de razonamiento y formato de salida esperado. Con esto, se busca que el modelo interiorice no solo la sintaxis, sino el estilo de razonamiento y planificación que debe seguir, reduciendo así la tasa de error en la generación de consultas complejas.

4.5.2. Generación robusta y autocorrección

La salida de un LLM no garantiza ni un formato bien formado ni una consulta correcta, por lo que el módulo no puede limitarse a extraer y ejecutar lo primero que el modelo devuelve. Durante la fase de validación se identificaron varios modos de fallo distintos que, de no tratarse de forma diferenciada, degradaban el comportamiento del sistema, y que motivaron consolidar la generación y la ejecución en un único mecanismo.

En primer lugar, la extracción de la consulta se diseñó para ser tolerante a respuestas imperfectas. El sistema localiza el bloque de código SQL aunque este no se haya cerrado correctamente, situación habitual cuando la respuesta se trunca por longitud, y dispone de un mecanismo de respaldo que reconoce tanto consultas que comienzan por `SELECT` como aquellas que se apoyan en expresiones de tabla común (`WITH`). Esta segunda consideración no es accesoria: las consultas analíticas de series temporales propias de este dominio recurren con frecuencia a *Common Table Expression (CTE)* y a funciones de ventana, por lo que un extractor que

solo contemplase sentencias `SELECT` descartaría por completo una parte sustancial de las consultas legítimas.

Sobre esa extracción se construye un bucle de autocorrección de hasta cinco intentos que distingue tres situaciones. Una respuesta truncada (un bloque de código abierto pero no cerrado) y una respuesta de la que no es posible extraer ninguna consulta ejecutable se tratan como motivos para solicitar una nueva generación acompañada de una indicación correctiva. Un error devuelto por DuckDB ante una consulta sintácticamente extraíble pero inválida —por ejemplo, una referencia a una columna que no existe en el esquema— se trata reinyectando la traza del error junto con la consulta que lo provocó en el flujo de pensamiento del agente, de modo que el modelo analice la discrepancia y genere una versión corregida de forma autónoma, emulando el proceso de depuración de un programador humano. La violación de la capa de seguridad constituye, en cambio, una parada sin reintento, por los motivos que se detallan en la sección 4.5.3.

Para asegurar un comportamiento determinista, el modelo se ejecuta a temperatura cero. Bajo esta configuración, un mismo prompt produce siempre la misma respuesta, por lo que la indicación correctiva debe modificar el prompt en cada reintento; de lo contrario, ante una respuesta truncada el sistema obtendría exactamente la misma salida truncada en los cinco intentos. Por ello, la retroalimentación se inyecta siempre, también en los casos en los que no existe una consulta previa que mostrar al modelo, garantizando que los tres caminos de fallo convergen en un mecanismo de corrección efectivo.

Cuando se agotan los intentos sin obtener un resultado válido, el módulo no devuelve una respuesta aproximada, sino que envía un error controlado. La gestión de esta situación recae en el orquestador, que en el modo híbrido renuncia explícitamente a la evidencia empírica y fundamenta la respuesta únicamente en la literatura, tal como se describe en la sección dedicada a dicho componente. De este modo, un fallo no recuperable del módulo SQL nunca se traduce en una respuesta que aparente estar fundamentada en datos que en realidad no se obtuvieron.

4.5.3. Capa de seguridad (*guardrails*)

El diseño incorpora una capa de validación previa a la ejecución que actúa como barrera de seguridad y garantiza que el módulo siempre opere en modo de solo lectura, preservando la integridad de los datos originales con independencia de lo que el LLM haya generado. Antes de enviar cualquier consulta a DuckDB, el sistema comprueba que se trate de una sentencia de solo lectura —una consulta `SELECT`, eventualmente precedida de expresiones de tabla común— y verifica la

ausencia de comandos destructivos o de alteración de datos, como `DROP`, `INSERT`, `ALTER`, `TRUNCATE`, `DELETE` o `UPDATE`.

Esta verificación no se realiza por coincidencia de subcadenas, sino analizando la sentencia por unidades léxicas tras eliminar los comentarios y los literales entre comillas. El motivo es que una comprobación por subcadena produce falsos positivos sobre construcciones legítimas: una columna denominada `last_update` o un filtro que compara contra el valor `'DELETED'` contienen las cadenas `update` y `delete` sin que la consulta deje de ser de solo lectura. La tokenización resuelve el primer caso, ya que el guion bajo forma parte del propio identificador y `last_update` se conserva como un único *token*; la limpieza previa resuelve el segundo, retirando el contenido entrecomillado antes de que pueda confundirse con una palabra reservada. Esta precisión resulta especialmente relevante porque, a diferencia del resto de fallos contemplados en la sección 4.5.2, una violación de seguridad es una parada que aborta la consulta sin reintento: intentar "corregir" una instrucción potencialmente insegura no es un comportamiento deseable, por lo que el coste de un falso positivo no recuperable obliga a que la validación sea precisa.

4.5.4. Trazabilidad

Tras superar las validaciones previas, se extraen los nombres de las tablas implicadas en la consulta, lo que permite al orquestador adjuntar a la respuesta final la trazabilidad de las fuentes de datos consultadas. Si DuckDB devuelve un error porque la consulta es sintácticamente extraíble pero inválida, la consulta entra de nuevo en el bucle de reintentos descrito en la sección 4.5.2 y la variable de trazabilidad se reinicia para no arrastrar nombres de tablas que no llegaron a consultarse con éxito.

4.6. Módulo RAG

Este módulo implementa una arquitectura RAG encargada de extraer, procesar y recuperar información desde un corpus de documentos en formato PDF para proporcionar contexto al LLM. Para la persistencia y recuperación de los vectores se utiliza ChromaDB, cuya elección se justifica en la sección 4.2.2.

4.6.1. Extracción y procesamiento

El proceso de extracción continúa con la lectura de los documentos. Para poder indexar la información correctamente, el texto extraído se divide en fragmentos o *chunks* de 1000 caracteres, aplicando un solapamiento de 200 caracteres entre ellos.

Este solapamiento se utiliza para mantener la continuidad semántica y no perder el contexto entre párrafos adyacentes. Después, cada fragmento se transforma en un vector numérico utilizando el modelo de embeddings `multilingual-e5-small`.

Además, para que los chunks recuperados sean de calidad, se aplican dos filtros en cascada antes de añadir cada fragmento a la base de datos vectorial:

- **Deduplicación por identificador:** A cada fragmento se le asigna un identificador único con el formato `fuente:pagina:indice`, donde el índice numera los chunks dentro de una misma página. Antes de insertar un fragmento, el sistema comprueba si su identificador ya existe en ChromaDB, lo que permite reprocesar el corpus sin generar duplicados.
- **Filtrado de bibliografía y referencias:** Mediante expresiones regulares se identifican las líneas que contienen patrones típicos de citas, concretamente nombres seguidos de un año entre paréntesis, identificadores DOI o direcciones URL. Para reducir falsos positivos, el patrón exige al menos diez caracteres antes del año, lo que evita que menciones triviales como "en 2023" activen el filtro. El fragmento se descarta únicamente cuando más de la mitad de sus líneas presentan estos patrones, de modo que un párrafo de contenido sustantivo con una cita esporádica se conserva, mientras que un bloque dominado por referencias se elimina.

4.6.2. Recuperación de información

Por último, el objetivo es recuperar el contexto más relevante posible para enviarlo al LLM. Para ello, se utiliza una técnica conocida como *over-fetching*. Esta consiste en que, en lugar de extraer directamente los k fragmentos necesarios, el sistema recupera inicialmente el doble ($2k$).

Sobre este conjunto ampliado se aplican dos criterios de selección de forma entrelazada. En primer lugar, se descartan aquellos fragmentos cuya distancia coseno respecto a la consulta es igual o superior al umbral configurado. En segundo lugar, se realiza una deduplicación por contenido textual, que elimina fragmentos idénticos procedentes de páginas distintas u operaciones de ingesta repetidas. El sistema acepta los candidatos en orden de similitud creciente hasta alcanzar los k fragmentos solicitados o agotar el conjunto del over-fetching, lo que ocurra antes. En el caso poco frecuente de que ni siquiera los $2k$ candidatos iniciales basten para reunir k fragmentos válidos, el módulo devuelve los que haya conseguido reunir.

Como detalle de implementación derivado de la convención de prefijos del modelo de embeddings (sección 4.8.2), cada fragmento se almacena en ChromaDB

precedido del prefijo `passage:`, mientras que la consulta del usuario se proyecta con el prefijo `query:` antes de invocar la búsqueda. Esta asimetría es lo que permite explotar el entrenamiento contrastivo del modelo y se discute en detalle en la sección referida.

4.7. Orquestador

El orquestador es el módulo central del sistema y actúa como único punto de entrada para las consultas del usuario, asumiendo la responsabilidad de coordinar el resto de componentes sin implementar lógica de dominio propia. Esta arquitectura centralizada corresponde a la explicada al inicio de este capítulo y responde al principio de separación de responsabilidades, ya que a este módulo no le interesa cómo están construidos internamente el resto de ellos, sino únicamente qué datos necesitan recibir y qué resultados devuelven. Gracias a esta abstracción, se consigue un sistema mantenible donde cualquier pieza puede ser modificada o sustituida sin afectar a la arquitectura global, siempre y cuando se respeten dichas interfaces de comunicación.

Al ejecutarse, inicializa las dependencias de los módulos (como DuckDB y ChromaDB) y delega las rutinas de preparación necesarias para su funcionamiento, tales como la ingesta del directorio de documentos para el RAG o la creación de vistas para DuckDB. Una vez preparado el entorno, espera la llegada de una consulta del usuario y la redirige al clasificador de intención. En función de la categoría detectada, demanda a cada módulo la información en bruto que necesita: por un lado, al módulo SQL le solicita el DataFrame resultante de la consulta a la base de datos y, por otro, al módulo RAG el contexto relevante para satisfacer la consulta y las fuentes consultadas.

Una de las decisiones arquitectónicas clave para que el sistema funcione de forma óptima siguiendo este estilo es que la generación de la respuesta final recae exclusivamente en el LLM del orquestador.

Esta centralización se justifica por dos motivos fundamentales:

- **Optimización de recursos y latencia:** Si cada módulo resolviese la consulta por su cuenta para que el orquestador unificase los textos a posteriori, se realizarían múltiples llamadas redundantes, lo cual duplicaría el coste computacional y aumentaría considerablemente el tiempo de espera del usuario.
- **Evitar la reinterpretación:** Cada vez que un LLM procesa y resume información, existe el riesgo de introducir sesgos, alucinaciones u omitir detalles

sutiles. Por tanto, si el orquestador recibiera resúmenes en lugar de datos crudos, estaría operando sobre información ya alterada. Al centralizar este proceso, los datos empíricos y la evidencia teórica llegan intactos al modelo final, garantizando la máxima fidelidad en la respuesta.

Por último, dependiendo de la clasificación de la consulta, el sistema opera en uno de los siguientes tres modos:

- **Modo SQL:** Solicita el DataFrame al módulo SQL y lo inyecta directamente en el prompt del LLM para generar una respuesta fundamentada en los datos empíricos.
- **Modo RAG:** Solicita el contexto relevante y las fuentes de donde proviene al módulo RAG, para luego inyectarlo en el LLM para formular una respuesta basada estrictamente en la literatura recuperada.
- **Modo híbrido:** Ejecuta ambos módulos y entrega toda la información en bruto al LLM, permitiendo que el modelo sintetice los datos empíricos y la literatura científica en una respuesta unificada y coherente.

Concretamente, en el modo híbrido si recibe un fallo no recuperable del módulo SQL —ya sea un error tras agotar los reintentos descritos en la sección 4.5.2 o un resultado vacío—, el orquestador no aborta la consulta ni oculta la incidencia. En su lugar, degrada explícitamente al modo RAG y antepone a la respuesta un aviso textual que informa al usuario de la ausencia de evidencia empírica y de que la respuesta se fundamenta exclusivamente en la literatura. Esta degradación controlada preserva la utilidad del sistema ante fallos parciales sin comprometer la transparencia sobre el origen de la información.

4.7.1. Diseño del prompt del modo RAG

El diseño del prompt se orientó de forma que induciese la propiedad de groundedness sobre la respuesta del modelo, entendida como el grado en que sus afirmaciones se derivan exclusivamente de la evidencia recuperada [4, 10]. Esta propiedad, junto con la context relevance evaluada sobre la estructura de recuperación, conforma el conjunto de métricas sobre las que se construye la validación experimental del módulo (sección 5.3). Los mecanismos a través de los cuales se persigue inducirla se desarrollan a continuación.

Aislamiento estructural de la pregunta y el contexto

La pregunta del usuario y el contexto recuperado se inyectan en el prompt encerrados entre delimitadores explícitos (`«<PREGUNTA»>`, `«<CONTEXTO_CIENTIFICO»>`)

que los separan tanto entre sí como del bloque de instrucciones. Esta separación estructural permite al modelo identificar con claridad qué parte del prompt es la pregunta del usuario, cuál es la evidencia sobre la que debe apoyarse y cuáles son las instrucciones que regulan su comportamiento.

Restricción a la evidencia recuperada

El prompt restringe explícitamente al modelo a responder únicamente con información presente en el contexto, prohibiendo el uso de los conocimientos previos adquiridos durante su entrenamiento. Esta restricción constituye el mecanismo principal sobre el que descansa la propiedad anunciada al inicio de la sección.

Plantilla de abstención ante ausencia de cobertura

Cuando el contexto recuperado no responde a la pregunta, el prompt obliga al modelo a emitir una respuesta de abstención con una plantilla textual fija que explicita qué se preguntó y de qué trata realmente el contexto disponible. Esta directiva extiende la groundedness a su caso límite: ante la ausencia de evidencia que sustente una respuesta, la respuesta correcta no es una reformulación plausible sino la declaración explícita de esa ausencia. La plantilla cumple además una función diagnóstica, ya que un fallo del *retriever* se manifiesta de forma legible en la respuesta del sistema y facilita la depuración del pipeline en su conjunto.

Filtrado residual de referencias bibliográficas

Como segunda barrera frente a un fallo del filtrado (sección 4.6.1), el prompt instruye al modelo para que ignore explícitamente cualquier fragmento del contexto que reconozca como una lista de referencias bibliográficas. De este modo, un *chunk* residual de bibliografía que hubiera escapado durante la fase de *chunking* no contamina la respuesta final.

4.7.2. Adaptación del prompt según la forma del resultado SQL

En el modo SQL, la estructura del DataFrame devuelto por el módulo SQL varía considerablemente según la consulta: una agregación produce una única fila, una serie temporal produce una secuencia ordenada por fecha y una consulta de filtrado puede producir desde una tabla pequeña hasta miles de registros. Inyectar todas estas formas con el mismo prompt genera respuestas inadecuadas: un único valor agregado tratado como tabla se transcribe en lugar de explicarse, y una tabla

extensa se resume con riesgo de omisiones o invenciones.

Para evitarlo, el orquestador clasifica el resultado en cuatro tipos (`AGREGACION`, `SERIE_TEMPORAL`, `TABLA_PEQUEÑA` y `TABLA_GRANDE`) y selecciona una instrucción específica para cada uno. La distinción entre serie temporal y tabla pequeña se realiza inspeccionando los tipos de las columnas en busca de campos de fecha u hora, y el umbral entre tabla pequeña y grande se fija en quince filas, valor que delimita el punto a partir del cual el LLM comienza a comprometer la fidelidad de la respuesta en base a los datos.

Un caso particular es el de `TABLA_GRANDE`: cuando el resultado supera el umbral de filas susceptibles de ser interpretadas con fidelidad, el orquestador devuelve la tabla directamente en formato Markdown sin invocar al LLM. Esta decisión se fundamenta en el mismo principio que motiva la centralización de la generación: cada paso de reinterpretación de un modelo de lenguaje introduce riesgo de alucinación, y resumir cientos o miles de registros excede la capacidad de un modelo local de tamaño moderado sin garantías de exactitud. Además, por encima de mil filas la tabla se trunca y se notifica al usuario el número de registros omitidos.

Esta adaptación por tipos no se aplica en el modo híbrido, donde el DataFrame se inyecta siempre con un único formato tabular y únicamente se conserva el truncado por encima de mil filas. La razón es que la tabla deja de ser el resultado final que el modelo debe comunicar al usuario y pasa a ser una pieza de evidencia empírica que se contrasta con la literatura, de modo que la instrucción dominante es la síntesis entre ambas fuentes y no la forma específica del resultado tabular.

4.8. Detalles de implementación

El diseño descrito a lo largo de este capítulo se materializó en un conjunto de módulos de Python desacoplados. Mientras que los detalles de realización de cada componente se han ido recogiendo junto a su diseño en las secciones anteriores, existen aspectos de implementación que son transversales a varios módulos y que, por tanto, no corresponden a ninguno de ellos en particular. Esta sección reúne dichos aspectos.

4.8.1. Organización del código y entorno de ejecución

La arquitectura modular y desacoplada justificada en la sección 4.1 tiene un reflejo directo en la organización del código: cada componente lógico se corresponde con un único fichero y una única clase. El clasificador de intención se encuentra

en la clase `ClasificadorIntencion`, el módulo SQL en `ConsultaSQL`, el módulo RAG en `GestorRAG` y el orquestador en `Orquestador`. Cada clase se expone de modo que el orquestador no necesita conocer la implementación interna del resto, sino únicamente qué información debe entregar y qué resultado recibe, en coherencia con el principio de separación de responsabilidades expuesto en la sección 4.7. El código completo del sistema, junto con las instrucciones de instalación y ejecución, se encuentra disponible públicamente en el repositorio del proyecto [6].

Todo el sistema se ejecuta de forma local, sin dependencia de servicios externos, en consonancia con los criterios de privacidad y coste computacional que motivaron las decisiones tecnológicas de la sección 4.1. Los LLM se sirven mediante `Ollama` sobre la máquina local, el modelo de embeddings opera íntegramente en CPU y la base de datos vectorial persiste en un directorio del propio sistema de ficheros. No se realiza ninguna llamada a API remotas ni se requiere GPU en ningún punto del pipeline. Las versiones concretas de las bibliotecas y modelos empleados se fijaron de forma explícita para garantizar la reproducibilidad de los resultados presentados en el capítulo de experimentación.

4.8.2. Convención de prefijos del modelo de embeddings

El modelo de embeddings seleccionado en la sección 4.2.3 pertenece a la familia *E5*[20, 21], entrenada mediante aprendizaje contrastivo débilmente supervisado sobre pares de consulta y documento. Su particularidad relevante para la implementación es que condiciona la representación que genera en función del rol del texto, mediante un prefijo de instrucción que se antepone a la entrada. No se trata de una arquitectura de dos codificadores independientes, es decir, un único codificador compartido produce ambas representaciones, y es el prefijo el que determina en qué rol se proyecta el texto. El prefijo `query:` corresponde a una intención de búsqueda o pregunta corta, mientras que `passage:` corresponde a un fragmento de documento susceptible de ser recuperado. Dado que existen dos módulos del sistema que emplean embeddings y comparten el mismo modelo pero utilizándolo con convenciones distintas, merece la pena documentarlo.

Por un lado, en el clasificador de intención, la consulta entrante se compara contra un conjunto de ejemplos que son, a su vez, preguntas. Ambas naturalezas textuales son por tanto idénticas, lo que obliga a etiquetar los dos lados de la comparación con el mismo prefijo `query:`. Si los ejemplos del clasificador se marcaran como `passage:`, el modelo los representaría según el rol de documento a recuperar, lo que perjudicaría la alineación semántica de la comparación y, con ella, la similitud coseno sobre la que se decide la categoría. Por este motivo, los ejemplos y la consulta del usuario se etiquetan ambos con `query:`, asegurando que

se comparan en el mismo espacio de representación.

Por otro lado, en el módulo RAG la situación es la opuesta y sí se aprovecha la asimetría del modelo. A cada fragmento extraído de los documentos PDF se le antepone el prefijo `passage:` antes de indexarlo en ChromaDB, mientras que a la consulta del usuario que llega al módulo se le antepone `query:`. De este modo, cada texto se proyecta según su rol real —corpus de conocimiento a recuperar frente a intención de búsqueda—, que es precisamente el escenario para el que la familia E5 fue entrenada. Esta diferencia de uso entre ambos módulos, pese a compartir un mismo modelo, es la razón por la que la convención de prefijos se trata como un detalle de implementación global y no como una particularidad de un componente concreto.

5 Experimentos y resultados

Una vez integrada y ajustada la arquitectura completa, se llevó a cabo una fase de validación experimental orientada a medir de forma objetiva la calidad de los componentes que más condicionan el comportamiento del sistema. Coherentemente con la metodología iterativa descrita en la sección 3.2, la evaluación no se planteó como una prueba única al final del desarrollo, sino como el cierre de un proceso de ajuste en el que cada decisión de diseño se contrastó con datos antes de fijarse.

Se evaluaron de forma independiente el clasificador de intención y el módulo RAG. Esta elección no es arbitraria: el primero determina la ruta que sigue toda consulta, por lo que un error en esta etapa invalida el resto del pipeline; el segundo es el componente cuya calidad depende de forma más sensible de los datos y de los hiperparámetros, y por tanto el que más se beneficia de un análisis empírico detallado. El módulo SQL, en cambio, tiene una corrección de naturaleza binaria —la consulta se ejecuta y devuelve los datos esperados o no lo hace— por lo que su validación se apoyó en el bucle de autocorrección descrito en la sección 4.5.2 y en pruebas manuales sobre consultas representativas (sección 5.4).

5.1. Metodología de evaluación

Todas las pruebas se ejecutaron en local, en coherencia con las decisiones tecnológicas justificadas en la sección 4.2. Los scripts de evaluación y los conjuntos de prueba utilizados están disponibles en el repositorio del proyecto [6], lo que permite reproducir los resultados presentados en este capítulo.

Por un lado, para el clasificador de intención se recurrió a métricas estándar de clasificación supervisada: exactitud (accuracy) global, precisión, exhaustividad (recall) y F1 promediados por clase, además de la matriz de confusión. Sean VP_c, FP_c y FN_c los verdaderos positivos, falsos positivos y falsos negativos asociados a una clase c, y sea N el número total de muestras evaluadas. Las cuatro métricas se definen como:

$$\text{Exactitud} = \frac{\sum_c VP_c}{N} \tag{5.1}$$

$$\text{Precisión}_c = \frac{VP_c}{VP_c + FP_c} \qquad \text{Recall}_c = \frac{VP_c}{VP_c + FN_c} \tag{5.2}$$

$$\text{F1}_c = 2 \cdot \frac{\text{Precisión}_c \cdot \text{Recall}_c}{\text{Precisión}_c + \text{Recall}_c} \tag{5.3}$$

La exactitud ofrece una medida global del rendimiento, mientras que la precisión y la exhaustividad evalúan el comportamiento del clasificador por clase: la primera mide qué proporción de las predicciones de una clase son correctas, y la segunda qué proporción de las instancias reales de esa clase son efectivamente recuperadas por el sistema. El F1 combina ambas en su media armónica, penalizando los desequilibrios. Para obtener una medida agregada en escenarios multiclase se utiliza el promedio macro, que calcula la métrica para cada clase de forma independiente y promedia los resultados sin ponderar por la frecuencia de las clases, dando el mismo peso a todas ellas.

Por otro lado, para el módulo RAG el enfoque es distinto. No existe una única respuesta correcta contra la que comparar, por lo que se optó por una evaluación basada en un modelo juez (*LLM-as-judge*) [24], implementada en la clase `JuezRAG`. El juez, ejecutado también con `Llama 3` en local, valora cada interacción según dos dimensiones complementarias en una escala de 1 a 5: la relevancia del contexto (context relevance), es decir, si los fragmentos recuperados contienen información útil para la pregunta, y la fundamentación (groundedness), es decir, en qué medida la respuesta generada se apoya en el contexto sin inventar información.

Cabe destacar una limitación de este método, que consiste en que el juez pertenece a la misma familia de modelos que el generador y actúa como evaluador único, sin validación humana ni medida de acuerdo entre evaluadores, por lo que sus puntuaciones deben interpretarse como un indicador de tendencia y no como una verdad absoluta.

5.2. Evaluación del clasificador de intención

5.2.1. Diseño experimental

El clasificador se evaluó sobre un conjunto de prueba de 66 preguntas, balanceado en las tres categorías (22 `SQL`, 22 `RAG` y 22 `HIBRIDO`) y construido de forma

independiente al banco de ejemplos, de modo que las preguntas de test no coinciden con las que el clasificador utiliza como referencia. Para medir el impacto de la variedad del banco de ejemplos se contrastaron dos configuraciones: un conjunto reducido inicial, de 21 ejemplos, empleado en las primeras versiones del sistema, y el conjunto definitivo de 63 ejemplos descrito en la sección 4.4.

Sobre cada configuración se compararon cinco estrategias de decisión. Un caso aleatorio, que sirve como suelo de referencia; el vecino más cercano (1-NN); la media de similitud sobre todos los ejemplos de cada categoría (media total); la media sobre los k ejemplos más cercanos de cada categoría (media top-k), que es la estrategia descrita en la sección 4.4.1; y un voto por mayoría sobre los k vecinos globales (voto top-k). Adicionalmente, cada estrategia se evaluó con y sin el mecanismo de margen de empate definido en la sección 4.4.2, con el fin de aislar su contribución real.

5.2.2. Evaluación con empate

Selección del número de vecinos

El primer experimento determina el valor del hiperparámetro k, cuya justificación quedó pendiente en la sección 4.4.1. La Figura 5.1 muestra la exactitud global de las estrategias media top-k y voto top-k al variar k entre 1 y 10 sobre el banco de ejemplos definitivo. La estrategia de media presenta un comportamiento claro: la exactitud crece de forma marcada hasta $k = 4$, donde alcanza su máximo (92.4 %), y a partir de ahí se estabiliza en una meseta que apenas se mueve hasta $k = 10$. El voto por mayoría es sistemáticamente inferior y necesita valores de k más altos para acercarse, sin llegar a igualar el rendimiento de la media. Este resultado confirma el razonamiento de diseño expuesto en la sección 4.4.1: promediar las mejores puntuaciones de cada clase es más robusto que contar votos, porque la decisión depende del valor real de la similitud y no del número de ejemplos cercanos acumulados. El valor $k = 4$ se adopta, por tanto, no de forma arbitraria, sino por situarse en el máximo de la curva.

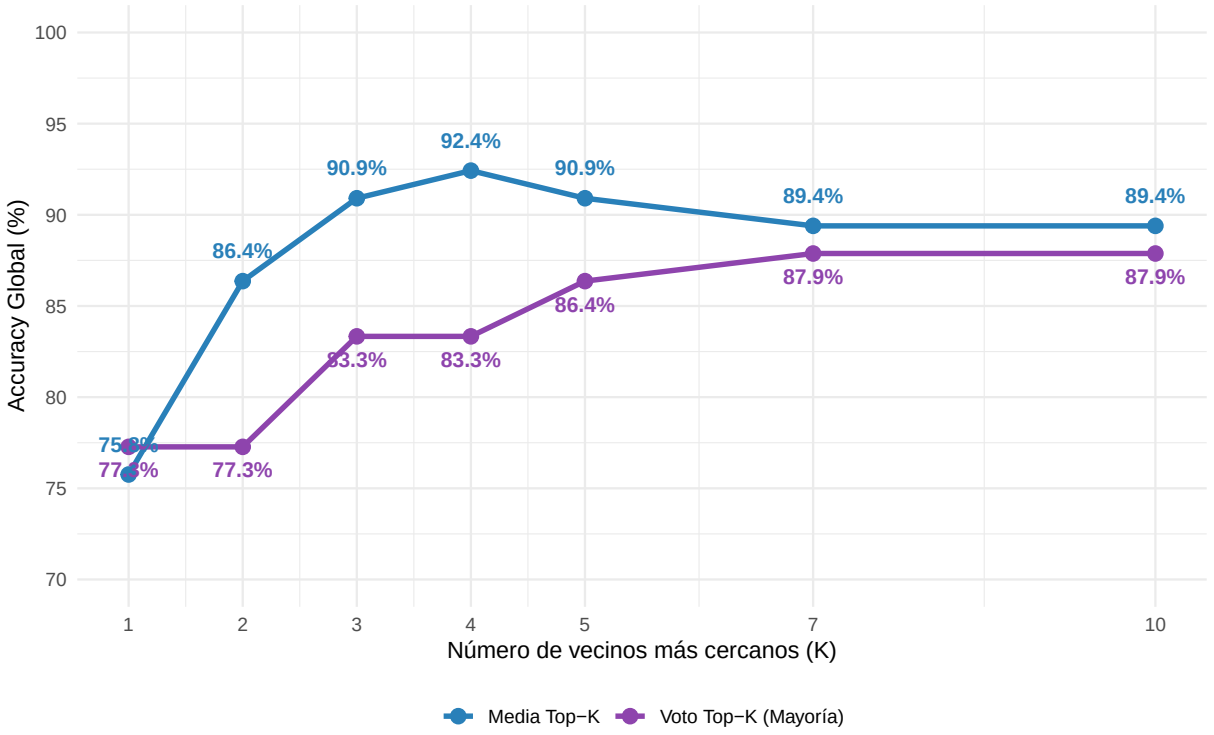


Figura 5.1: Evolución del parámetro k

Comparativa de estrategias

Fijado el hiperparámetro, la Figura 5.2 enfrenta las cinco estrategias con sus parámetros óptimos sobre el conjunto definitivo. El caso aleatorio se sitúa en torno al 30 %, un valor próximo al azar para tres clases equiprobables, lo que sirve para confirmar que el resto de medidas reflejan capacidad real de discriminación. El vecino más cercano alcanza un 77.3 %, la media total un 86.4 %, y la estrategia de media top-4 se impone con claridad sobre el resto, con una exactitud del 92.4 % y un F1 macro del 92.4 % al aplicar el margen de empate. Es relevante señalar que esta es exactamente la configuración desplegada en el sistema ("TOP_K = 4" junto con el margen), de modo que la elección de diseño y el óptimo empírico coinciden.

Conviene además señalar la consistencia interna de las métricas: en todos los clasificadores con capacidad real de discriminación las cuatro medidas se mueven de forma sólida, con la precisión ligeramente por encima del recall, lo que es indicio de un conjunto bien balanceado y valida el uso del F1 macro como criterio de comparación.

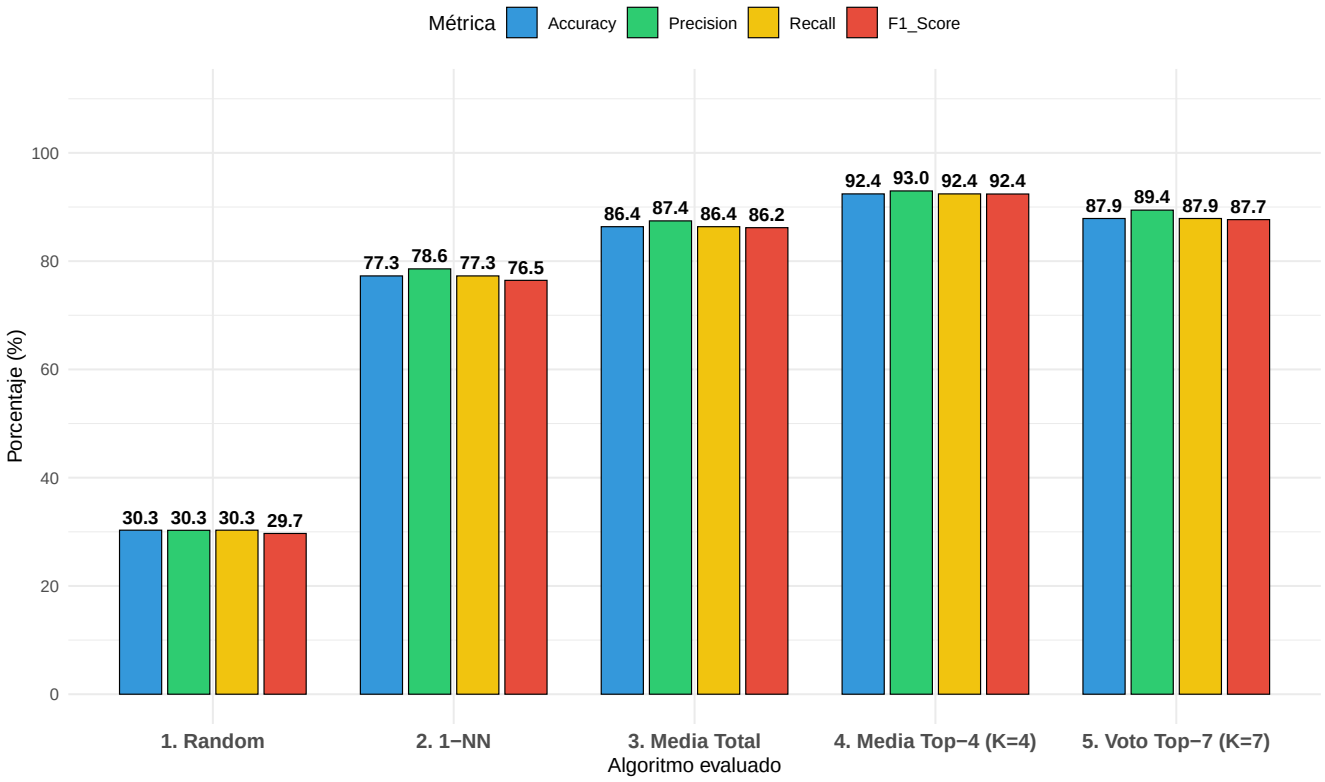


Figura 5.2: Métricas globales

Análisis de la configuración final

La matriz de confusión de la configuración ganadora, recogida en la Figura 5.3, permite descomponer ese 92.4 % global por categorías sobre un conjunto de evaluación perfectamente balanceado (22 preguntas por clase, 66 en total). De las 22 preguntas SQL, las 22 se clasifican correctamente; de las 22 RAG, 19 se aciertan, con 1 derivada a SQL y 2 a HÍBRIDO; y de las 22 HÍBRIDO, 20 se identifican bien y 2 se desvían a SQL.

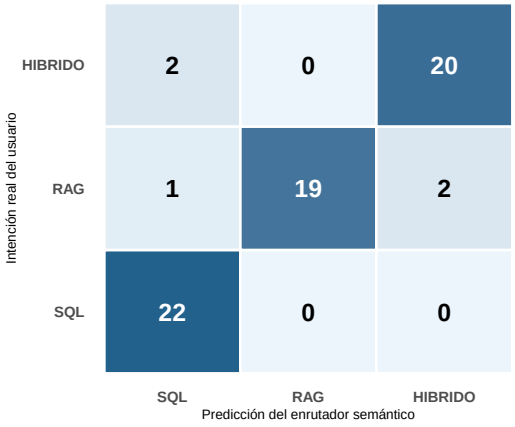


Figura 5.3: Matriz de confusión

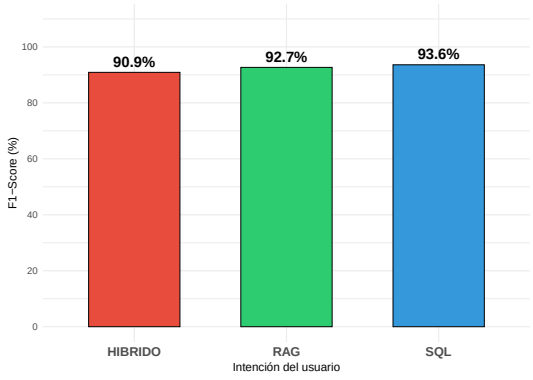


Figura 5.4: F1-Score por clase

Esta distribución de aciertos y fallos se traduce directamente en las métricas de precisión y recall de cada clase, cuya media armónica determina el F1 mostrado en la Figura 5.4. La Tabla 5.1 recoge dicha descomposición.

Clase	Precisión	Recall	F1
SQL	88 %	100 %	93.6 %
RAG	100 %	86.4 %	92.7 %
HÍBRIDO	90.9 %	90.9 %	90.9 %

Tabla 5.1: Precisión, recall y F1 por clase derivados de la matriz de confusión.

Desglosar las métricas por clase permite observar matices que el accuracy global oculta. Por ejemplo, la ruta SQL consigue un recall del 100 % al capturar todas sus consultas sin dejar escapar ninguna. Sin embargo, su precisión baja al 88 % porque absorbe 3 falsos positivos (uno de RAG y dos de `HIBRIDO`). En la práctica, esto indica que, ante distancias muy cortas en el espacio vectorial, el cálculo de similitudes tiende a decantarse por la jerga de SQL. La clase RAG muestra el comportamiento contrario: el agrupamiento semántico es mucho más estricto.

Cuando una consulta se asocia a esta intención por proximidad, no hay errores (precisión del 100 %), pero a cambio pierde 3 consultas reales de RAG que acaban desviándose a las otras ramas por alejamiento semántico (recall del 86.4 %).

Respecto a la clase `HIBRIDO`, su precisión y su recall empatan en un 90.9 %. Es un resultado lógico dentro de la arquitectura: al ser una categoría que ocupa la frontera en el espacio de embeddings entre las consultas estructuradas y las documentales, es esperable que sus confusiones se solapen de forma natural con ambas. Globalmente, los valores de F1 se mantienen por encima del 90 % en todas las categorías, lo que confirma que el orquestador no degrada una ruta para favorecer a otra. Aunque el F1 macro coincida numéricamente con el accuracy (92.4 %) debido a que el conjunto de evaluación está perfectamente balanceado, este análisis demuestra que la fiabilidad del enrutador varía según el contexto: la proximidad vectorial asegura la cobertura total cuando se trata de consultas estructuradas, pero exige una coincidencia mucho más exacta en el espacio latente antes de desviar el flujo hacia la recuperación de documentos (RAG).

5.2.3. Evaluación sin empate

Selección del número de vecinos

Primero se determina el valor del hiperparámetro k en ausencia del margen de empate. La Figura 5.5 muestra la exactitud global de las estrategias media top-k y voto top-k al variar k entre 1 y 10. La estrategia de media presenta un comportamiento semejante a su configuración con margen, pero ligeramente diferente: crece de forma rápida hasta $k = 4$, donde alcanza su máximo (90.9 %), y a partir de ahí se estabiliza manteniendo el mismo accuracy. Por su parte, el voto por mayoría vuelve a ser sistemáticamente inferior. Cabe destacar que la presencia o ausencia del margen de empate no ejerce ninguna influencia sobre este método —cuyos resultados son idénticos en ambos escenarios—, ya que su mecanismo de decisión ignora las magnitudes de similitud y se limita a contabilizar la categoría con mayor número de representantes. En consecuencia, sigue necesitando valores de k más altos ($k = 7$) para alcanzar su propio techo (87.9 %), sin llegar a igualar a la media.

Este resultado ratifica que, independientemente del uso de márgenes, promediar las mejores puntuaciones de cada clase sigue siendo estructuralmente más robusto que el recuento de votos. El valor $k = 4$ se mantiene, por tanto, como el óptimo técnico de la arquitectura.

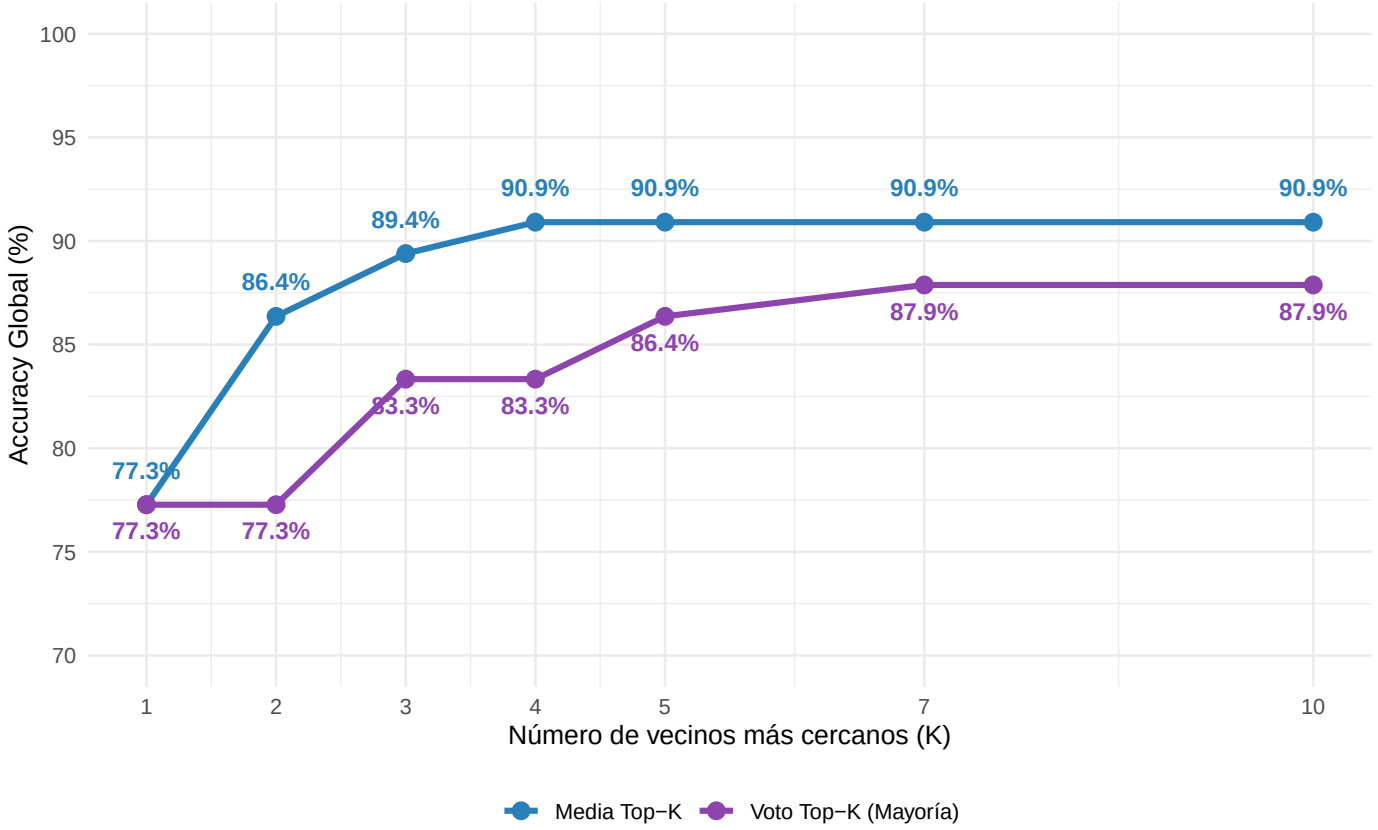


Figura 5.5: Evolución del parámetro K sin margen de empate

Comparativa de estrategias

Fijado el hiperparámetro, la Figura 5.6 enfrenta las cinco estrategias con sus parámetros óptimos. El caso aleatorio se mantiene estable en torno al 36.4 %, confirmando que las métricas superiores reflejan un aprendizaje semántico real. El vecino más cercano (1-NN) se detiene en un 77.3 % y la media total en un 87.9 %. Nuevamente, la estrategia de media top-4 se impone con claridad, logrando una exactitud del 90.9 % y un F1 macro del 90.9 %.

Al igual que en la configuración ganadora, las métricas muestran consistencia interna: la precisión general de la media top-4 (91.5 %) se sitúa ligeramente por encima de su recall (90.9 %), validando la calidad del espacio dimensional. Sin embargo, la caída del accuracy global desde el 92.4 % (con empate) hasta el 90.9 % (sin empate) anticipa que la eliminación del margen penaliza el rendimiento general del sistema.

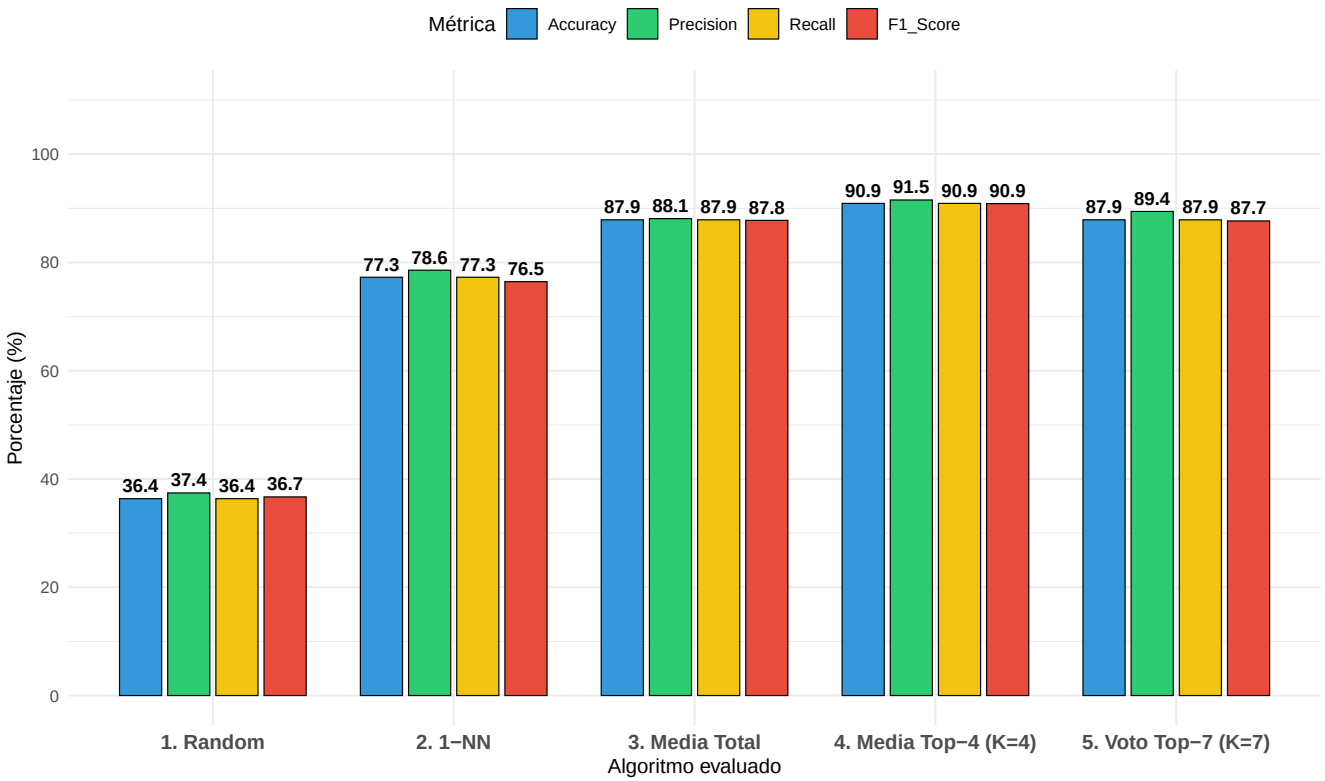


Figura 5.6: Métricas globales sin margen de empate

Análisis de la configuración final

La matriz de confusión de esta configuración, recogida en la Figura 5.7, permite diseccionar ese 90.9 % global sobre el mismo conjunto de evaluación balanceado (22 preguntas por clase, 66 en total). De las 22 preguntas SQL, las 22 se clasifican correctamente; de las 22 RAG, 19 se aciertan, con 2 derivadas a SQL y 1 a `HÍBRIDO`; y de las 22 `HÍBRIDO`, 19 se identifican bien, con 2 desviadas a SQL y 1 a RAG.

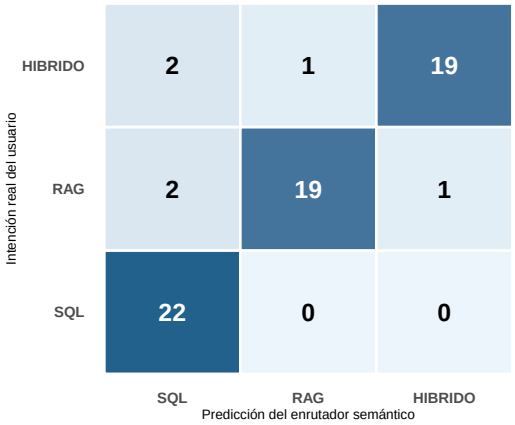


Figura 5.7: Matriz de confusión (sin empate)

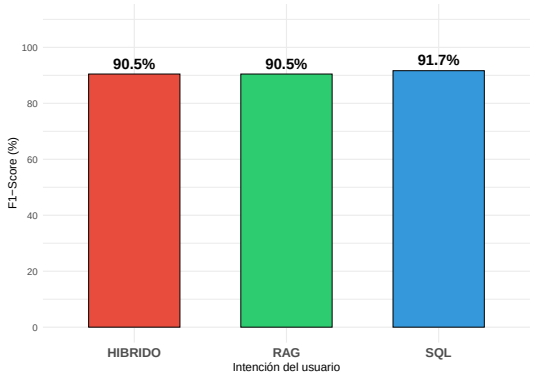


Figura 5.8: F1-Score por clase (sin empate)

Esta configuración altera el equilibrio de aciertos y fallos respecto a la configuración con margen. La Tabla 5.2 recoge los valores obtenidos.

Clase	Precisión	Recall	F1
SQL	84.6 %	100.0 %	91.7 %
RAG	95.0 %	86.4 %	90.5 %
HÍBRIDO	95.0 %	86.4 %	90.5 %

Tabla 5.2: Precisión, recall y F1 por clase en ausencia del margen.

El análisis comparativo de estas figuras evidencia un cambio en el comportamiento del sistema. Al suprimir el umbral de tolerancia, la clase SQL se agudiza como ruta "por defecto" y foco principal de errores. Aunque mantiene su recall perfecto del 100 % (ninguna consulta SQL se escapa), su precisión se desploma al 84.6 % (cayendo desde el 88 %): de las 26 veces que el clasificador predice SQL, 4 son falsos positivos. Esto evidencia que, ante la ambigüedad, el clasificador tiende a asignar valores de similitud ligeramente superiores a la jerga estructurada.

Por otro lado, al suprimir el margen, las clases RAG e `HIBRIDO` pasan a exhibir un patrón numérico idéntico (95.0 % de precisión y 86.4 % de recall), pero llegan a él por motivos distintos. La clase RAG demuestra ser estructuralmente estable: mantiene intacto su recall del 86.4 % (sigue dejando escapar tres instancias, igual que con empate), pero pierde su precisión perfecta (cae del 100 % al 95.0 %) al absorber erróneamente una consulta. Por su parte, la clase `HIBRIDO` se vuelve más restrictiva: su precisión se eleva al 95.0 % (cuando el sistema predice esta ruta, su grado de certeza es casi absoluto), pero su recall cae al 86.4 % al fugar instancias que acaban en la ruta SQL.

Este estudio empírico justifica la arquitectura de enrutamiento diseñada. Sin el margen de empate, el clasificador padece de un sesgo que deriva preguntas contextuales y mixtas hacia la ejecución relacional. Podemos concluir que la inclusión del margen protege la frontera semántica de la clase `HIBRIDO` y consigue elevar la fiabilidad global del sistema del 90.9 % al 92.4 %.

5.2.4. Impacto del tamaño del banco de ejemplos

Para evaluar la sensibilidad del clasificador al volumen de ejemplos, se ha contrastado el rendimiento del sistema utilizando un banco reducido frente al banco definitivo analizado en las secciones anteriores. El objetivo es determinar si el clasificador es capaz de generalizar correctamente con unas pocas muestras (*Few-Shot Learning*) o si, por el contrario, el modelo es fuertemente dependiente de la densidad de los datos.

La comparativa global evidencia un resultado concluyente: la escasez de ejemplos penaliza severamente la capacidad discriminativa del modelo. La configuración elegida para el sistema media top-4, que alcanzaba un accuracy del 92.4 % con el banco definitivo, sufre un desplome, cayendo hasta un 72.7 %. De hecho, en este escenario la estrategia de `media total` (75.8 %) llega a superar ligeramente al truncamiento `top-4`, lo cual indica que ante la escasez de ejemplos ilustrativos no hay grandes diferencias entre ambos.

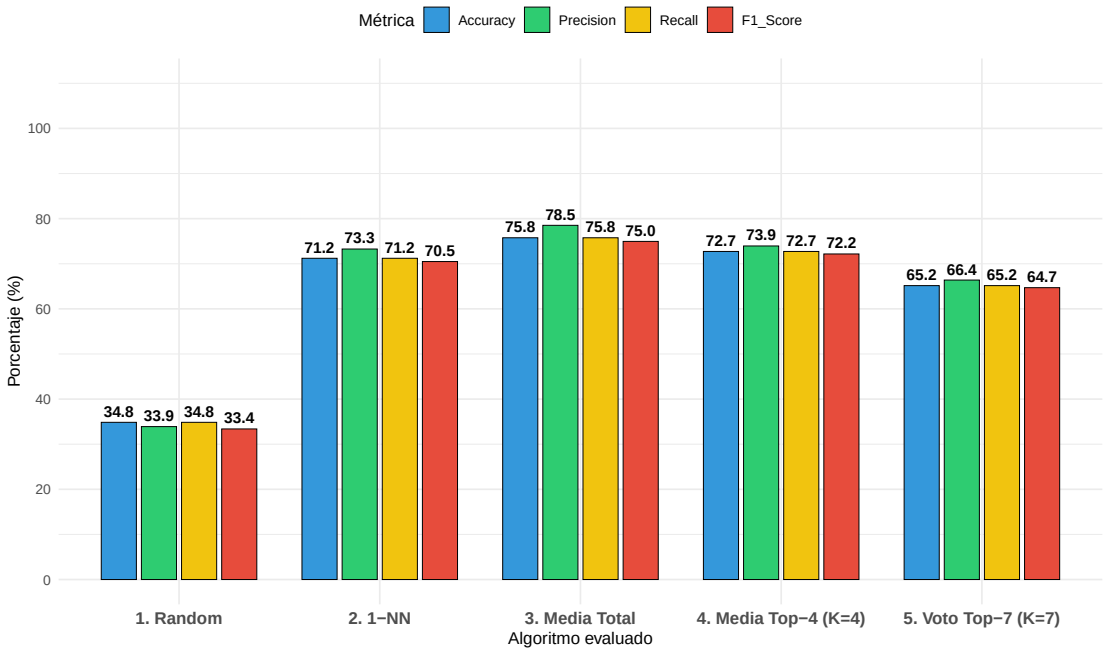


Figura 5.9: Métricas globales con banco reducido

Al analizar la matriz de confusión de esta configuración, recogida en la Figura 5.10, se observa que el impacto del déficit de datos altera el equilibrio de aciertos y fallos de forma asimétrica entre las categorías.

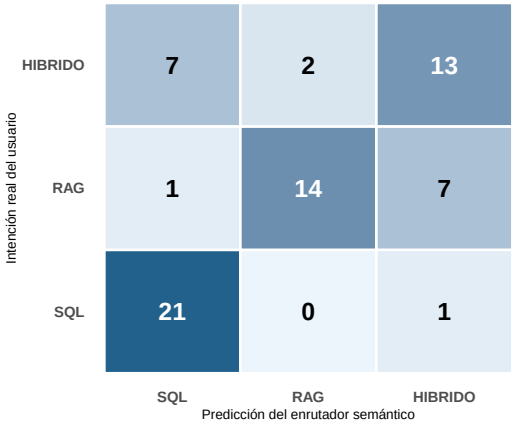


Figura 5.10: Matriz de confusión (banco reducido)

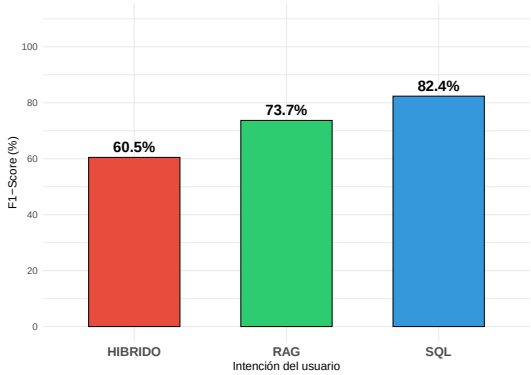


Figura 5.11: F1-Score por clase (banco reducido)

La Tabla 5.3 desglosa matemáticamente esta degradación, mostrando el retroceso generalizado de las métricas en comparación con el escenario óptimo.

Clase	Precisión	Recall	F1
SQL	72.4 %	95.5 %	82.4 %
RAG	87.5 %	63.6 %	73.7 %
HÍBRIDO	61.9 %	59.1 %	60.5 %

Tabla 5.3: Precisión, recall y F1 por clase con el banco de ejemplos reducido.

La clase SQL demuestra ser la más robusta ante la falta de ejemplos. Aunque su precisión cae al 72,4 % por absorber falsos positivos de las demás categorías, logra mantener su reconocimiento en un nivel muy elevado (95,5 % de recall, con 21 aciertos sobre 22), consolidando un F1-Score del 82,4 %. Las particularidades de la jerga de bases de datos estructuradas generan patrones lo suficientemente distanciados en el espacio dimensional como para ser detectados con pocas referencias, sufriendo apenas un único falso negativo hacia la ruta híbrida.

Por el contrario, el verdadero impacto del déficit de datos recae sobre las clases RAG e `HIBRIDO`, cuyas métricas se desploman, tal y como muestra la Figura 5.11. Destaca especialmente la clase `HIBRIDO`, cuya exhaustividad (recall) desciende hasta el 59,1 %. La causa de este desplome se aprecia claramente en la matriz de confusión, que revela un notable solapamiento entre categorías: 7 consultas de intención RAG son clasificadas erróneamente en la ruta `HIBRIDO`, mientras que otras 7 consultas de naturaleza `HÍBRIDO` se desvían hacia la ruta SQL. Esto demuestra que la escasez de muestras impide al modelo establecer un límite de decisión preciso entre las intenciones puramente documentales (RAG) y las mixtas (`HIBRIDO`).

5.3. Evaluación del módulo RAG

5.3.1. Diseño experimental

El módulo RAG se evaluó mediante el modelo juez descrito en la sección 5.1, instanciando Llama 3 a temperatura cero y forzando al evaluador a emitir su análisis dentro de la etiqueta `<razonamiento>` y la puntuación numérica dentro de `<puntuacion>`, con un parser que admite tanto la etiqueta cerrada como la abierta y, como último recurso, recupera el último entero del texto. Esta arquitectura del juez permite robustez frente a desviaciones puntuales del formato de salida.

Se probaron todas las combinaciones del producto cartesiano entre el número de fragmentos recuperados ($k \in \{3, 5, 7\}$) y el umbral de distancia coseno, barrido en dos rangos complementarios: uno amplio, de 0,175 a 0,65, diseñado para localizar el régimen útil, y otro fino, de 0,1 a 0,2, motivado por los hallazgos del primero y

orientado a caracterizar el rango de umbrales en el que el filtro descarta efectivamente parte de los candidatos. El resultado son $3 \times 10 = 30$ configuraciones por dataset.

Para someter al sistema a condiciones de dificultad realistas se emplearon dos conjuntos de 30 preguntas cada uno. El primero, que denominaremos `AUTORES`, está compuesto por preguntas con referencia explícita a un autor o estudio concreto (por ejemplo, "Según los ensayos de Mishra y Mishra (2012), ¿qué tipo de regresión o función matemática...?"), un caso especialmente exigente para la recuperación porque obliga al motor de búsqueda a localizar un documento bibliográfico específico. El segundo, denominado `SIMPLES`, está formado por preguntas de carácter general y conceptual sobre los mismos temas, sin referencias bibliográficas. Esta separación es deliberada y constituye una prueba de estrés que permite delimitar el rango de funcionamiento del sistema en lugar de medir un único escenario promedio.

El experimento completo se compone, por tanto, de 60 configuraciones y 1800 evaluaciones del juez (60 configuraciones $\times$ 30 preguntas).

5.3.2. Régimen del umbral: descubrimiento de la saturación

El primer resultado de las pruebas es más estructural que estadístico: los CSVs producidos para los umbrales $u \in \{0{,}25, 0{,}35, 0{,}45, 0{,}55, 0{,}65\}$ son idénticos para los tres valores de K en ambos conjuntos de datos. En la práctica, esto significa que el umbral deja de filtrar fragmentos una vez supera el valor de 0,25.

El método `similarity_search_with_score` de ChromaDB devuelve la distancia coseno, y el filtro `score <umbral_similitud` mantiene los fragmentos cuya distancia es menor que el umbral. La distribución empírica de distancias de los top-$2K$ (over-fetching 4.6.1) candidatos del modelo `multilingual-e5-small` sobre el corpus del Mar Menor nunca supera 0,25, por lo que cualquier umbral por encima de ese valor deja pasar todos los candidatos sin filtrar. Este resultado, lejos de ser una limitación del diseño experimental, constituye una observación empírica sobre la distribución de distancias del modelo de embeddings en el dominio considerado, y debe leerse como tal.

Por debajo de 0,25 se identifican tres regímenes diferenciados:

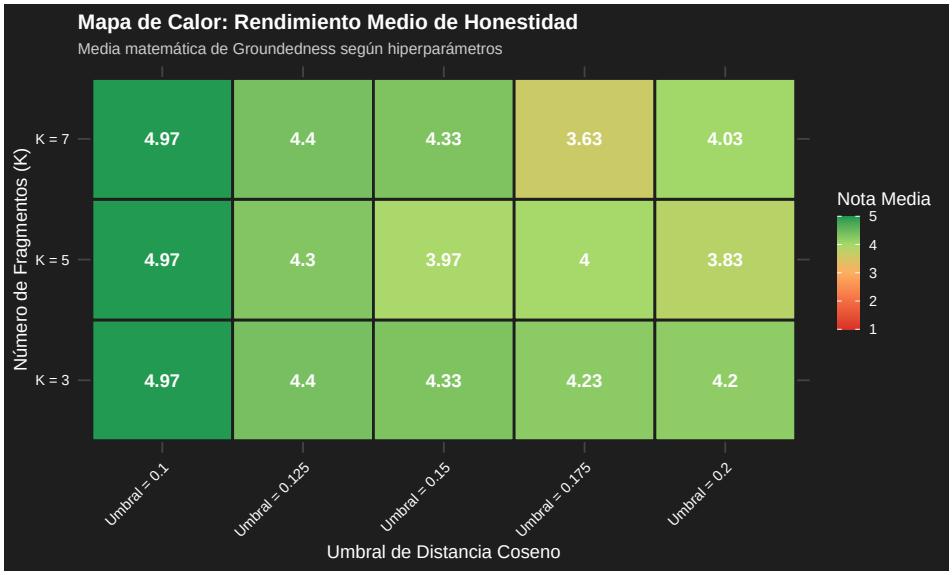
Tabla 5.4: Regímenes empíricos del umbral de distancia coseno.

Régimen	**Rango**	**Comportamiento**
Restrictivo	$u = 0{,}1$	El filtro descarta casi todos los candidatos; CR cae a 1,1 (`AUTORES`) y 1,6 (`SIMPLES`); GR sube a 4,97–5,00 por abstención sistemática.
Útil	$0{,}125 \leq u \leq 0{,}20$	El umbral filtra activamente; transición abrupta entre 0,1 y 0,125.
Saturado	$u \geq 0{,}25$	El umbral no filtra; sólo K tiene efecto.

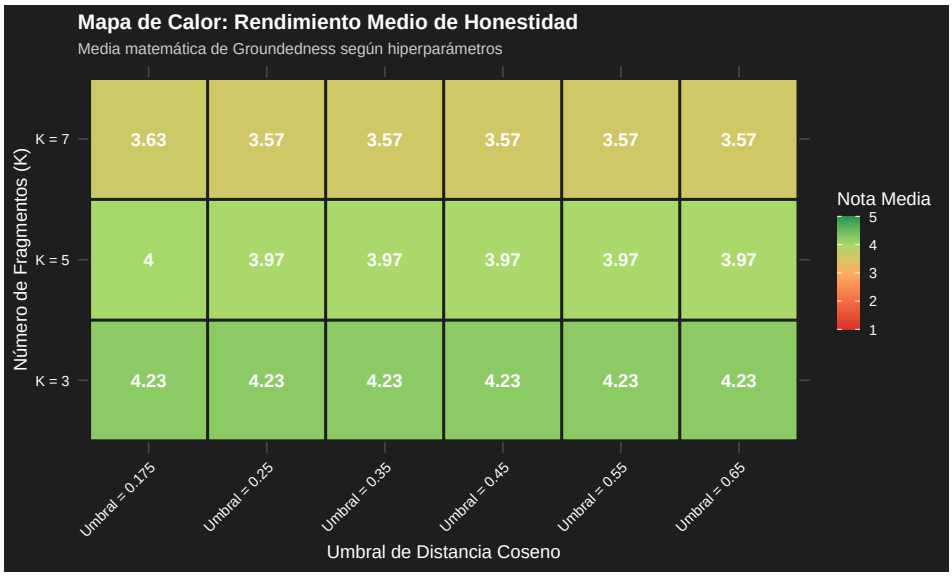
El régimen restrictivo merece una explicación adicional en cuanto a lo que supone en el sistema. En $u = 0{,}1$ el sistema rara vez produce contexto utilizable, lo que fuerza al LLM a invocar la instrucción de abstención ("No tengo información suficiente") por defecto, instrucción que el prompt de fundamentación premia con un 5 automático. El resultado es una combinación aparentemente favorable de groundedness alta y context relevance baja que en realidad refleja que la información no se entrega, se rechaza honestamente. Esto recuerda la importancia de evaluar siempre ambas métricas en conjunto.

La transición entre los diferentes regímenes se hace evidente al observar los mapas de calor de la Figura 5.12 (para `AUTORES`) y la Figura 5.13 (para `SIMPLES`). En las subfiguras 5.12a y 5.13a se aprecia con claridad la anomalía del umbral $u = 0{,}1$, donde la métrica de groundedness se dispara hasta el máximo debido a la abstención sistemática del modelo. A su vez, se constata visualmente un margen de ajuste efectivo entre 0,125 y 0,20, donde las puntuaciones oscilan, reflejando que se están filtrando fragmentos.

Por otro lado, las subfiguras 5.12b y 5.13b capturan la transición hacia la saturación: a partir de $u = 0{,}25$, las matrices se vuelven completamente uniformes por filas. Esto demuestra que cualquier incremento a partir de este punto en el umbral resulta inocuo para el proceso de recuperación, delegando todo el control del contexto en el parámetro K.



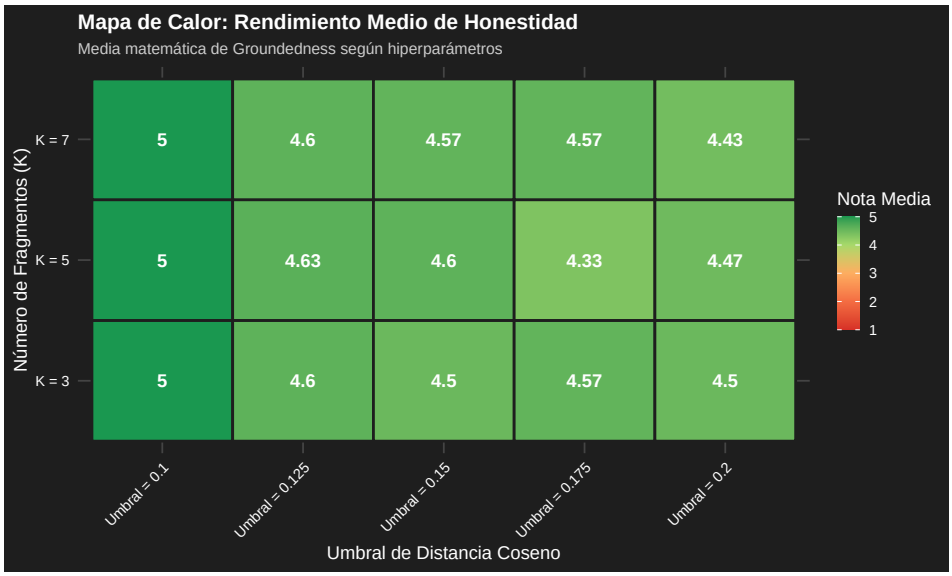
(a) Régimen restrictivo y útil ($u \in [0{,}1, 0{,}2]$).



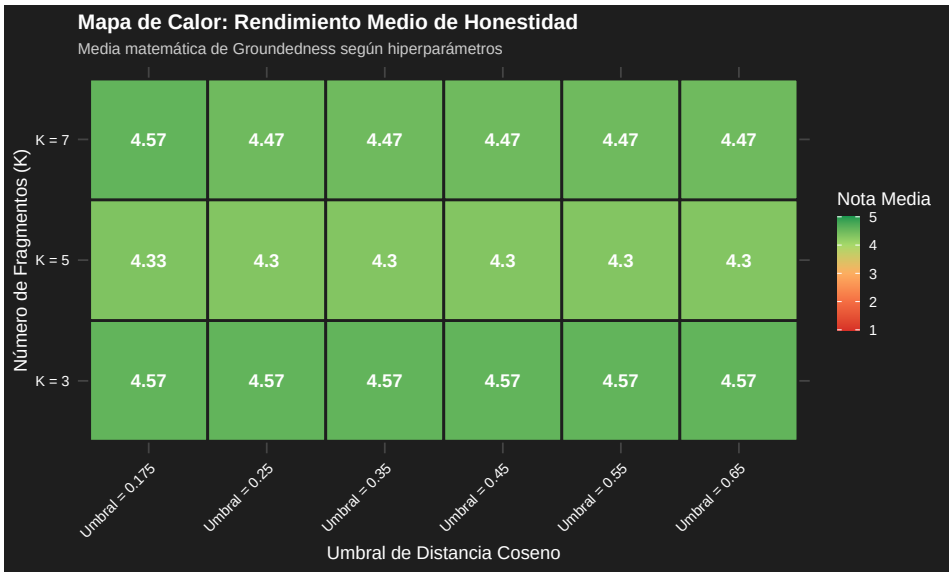
(b) Régimen saturado ($u \geq 0{,}25$) partiendo del límite útil ($u = 0{,}175$).

Figura 5.12: Mapas de calor de groundedness por (K, u) para el dataset `AUTORES`.

La consecuencia de este hallazgo es metodológicamente muy útil. Al aislar el umbral en la zona de saturación ($u = 0{,}25$), nos aseguramos de que el filtro del umbral queda inactivo, permitiéndonos evaluar el impacto puro del parámetro K (número de fragmentos) sobre las métricas del modelo sin interferencias.



(a) Régimen restrictivo y útil ($u \in [0{,}1, 0{,}2]$).



(b) Régimen saturado ($u \geq 0{,}25$) partiendo del límite útil ($u = 0{,}175$).

Figura 5.13: Mapas de calor de groundedness por (K, u) para el dataset `SIMPLES`.

5.3.3. Compromiso entre K y métricas

Para aislar el impacto del parámetro K (número de fragmentos) y entender el comportamiento base del sistema, se analizan primero los resultados en el régimen saturado ($u = 0{,}25$). Al estar el filtro de umbral inactivo en este punto, cualquier variación en las métricas se debe exclusivamente al volumen de texto inyectado en el prompt. Los resultados de esta línea base se resumen en la Tabla 5.5, que muestra las medias y el porcentaje de evaluaciones aceptables (puntuación ≥ 4).

Tabla 5.5: Resultados medios en el régimen saturado del umbral ($u = 0{,}25$).

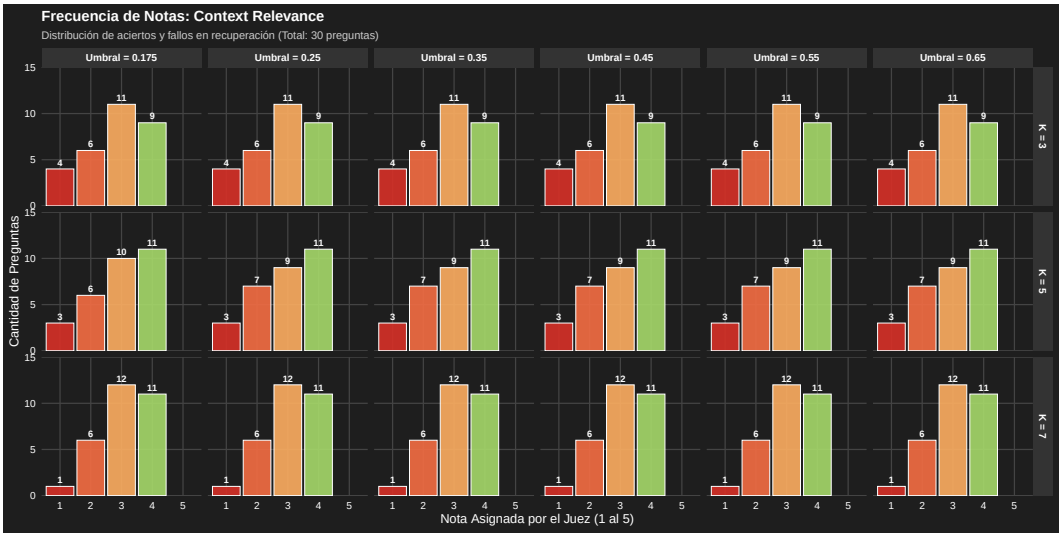
Dataset	K	CR media	GR media	CR ≥ 4	GR ≥ 4	GR $= 1$
AUTORES	3	2.833	4.233	30.0 %	76.7 %	0.0 %
AUTORES	5	2.933	3.967	36.7 %	73.3 %	6.7 %
AUTORES	7	3.100	3.567	36.7 %	60.0 %	13.3 %
SIMPLES	3	3.467	4.567	53.3 %	100.0 %	0.0 %
SIMPLES	5	3.600	4.300	70.0 %	90.0 %	0.0 %
SIMPLES	7	3.633	4.467	63.3 %	93.3 %	0.0 %

Los datos del dataset `AUTORES` muestran la tensión típica de los sistemas RAG entre recuperación y fundamentación. Al aumentar el número de fragmentos, la relevancia del contexto mejora levemente (la CR media sube de 2,83 $\rightarrow$ 2,93 $\rightarrow$ 3,10), pero la fundamentación empeora de forma notable (la GR media cae de 4,23 $\rightarrow$ 3,97 $\rightarrow$ 3,57). Añadir más fragmentos introduce contexto irrelevante o periférico, lo que confunde a Llama 3 y provoca que las alucinaciones graves (GR $= 1$) escalen del 0,0 % al 13,3 % al llegar a $K = 7$.

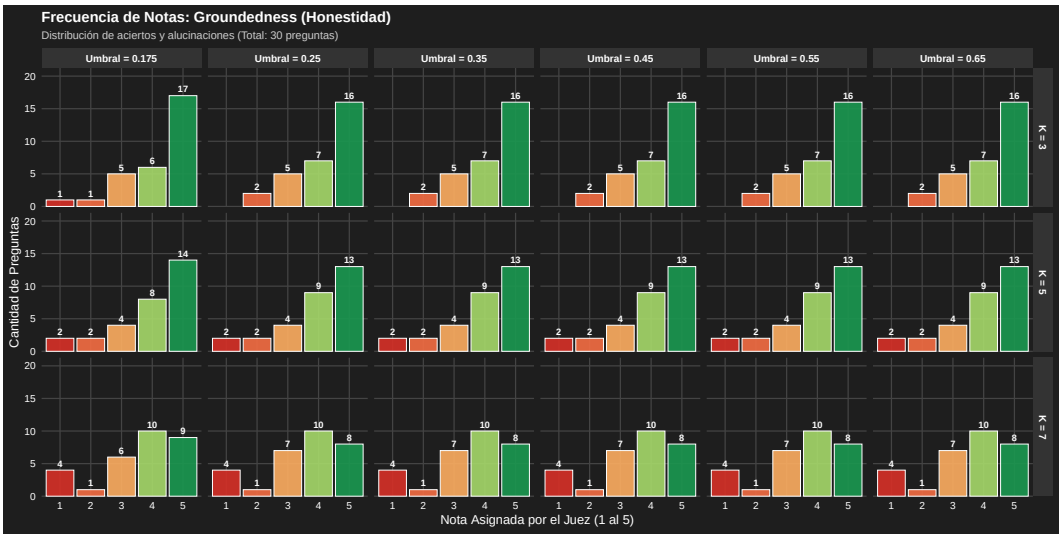
Además, la tasa de preguntas con éxito en recuperación (CR ≥ 4) se estanca en el 36,7 % al pasar de $K = 5$ a $K = 7$. Esto significa que ampliar la búsqueda a 7 fragmentos añade texto que aporta ruido (subiendo notas bajas de 2 a 3), pero no introduce información de alta calidad. En el dataset `SIMPLES`, el sistema es más robusto porque las preguntas son conceptuales y la información está concentrada en pocos fragmentos. Aquí, la configuración $K = 3$ destaca por lograr un 100,0 % de respuestas bien fundamentadas (GR ≥ 4), un perfil conservador útil si el único requisito crítico fuera la ausencia total de alucinaciones.

Las distribuciones de notas que se muestran en los gráficos de barras de la Figura 5.14 y la Figura 5.15 confirman visualmente este patrón.

Tomando los dos datasets en conjunto, $K = 5$ se impone como la elección

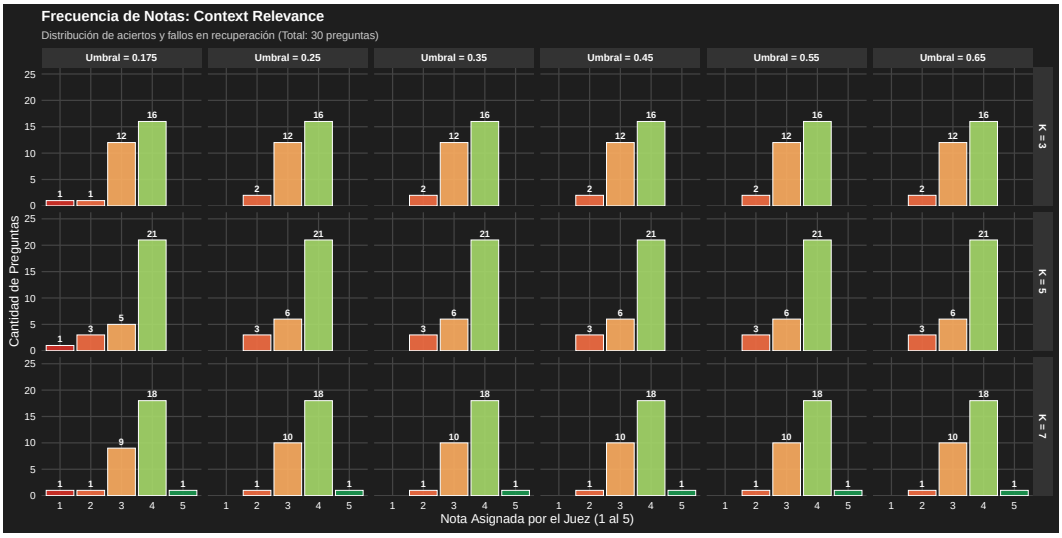


(a) Context relevance

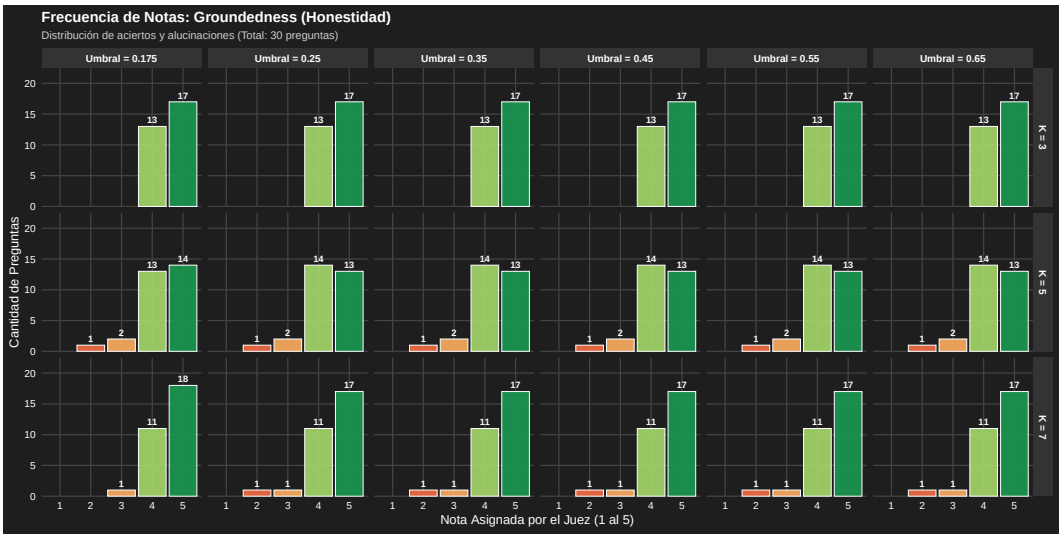


(b) Groundedness

Figura 5.14: Distribución de notas en el régimen saturado ($u \geq 0{,}25$) partiendo del límite útil ($u = 0{,}175$) para el dataset `AUTORES`.



(a) Context relevance



(b) Groundedness

Figura 5.15: Distribución de notas en el régimen saturado ($u \geq 0{,}25$) partiendo del límite útil ($u = 0{,}175$) para el dataset `SIMPLES`.

óptima. Iguala a $K = 7$ en cobertura sobre el dataset crítico `AUTORES` (36,7 % de preguntas con CR ≥ 4) y lo supera con claridad en `SIMPLES` (70,0 %, frente al 63,3 % de $K = 7$ y el 53,3 % de $K = 3$), manteniendo a la vez la mitad de alucinaciones graves que $K = 7$ en `AUTORES` (6,7 % frente al 13,3 %). Por otra parte, $K = 3$, pese a su nula tasa de alucinación, sacrifica entre 7 y 17 puntos porcentuales de cobertura según el dataset, un coste demasiado alto para un sistema cuya finalidad es entregar respuestas informativas. Se fija, por tanto, $K = 5$ como candidato a valor por defecto, pendiente de verificar que la elección se mantiene en el régimen útil del umbral.

5.3.4. Comportamiento en el régimen útil y cuello de botella

Antes de pasar al análisis de los umbrales que conforman el régimen útil, es necesario verificar si el valor $K = 5$ —seleccionado previamente a partir del régimen saturado— mantiene su validez cuando el filtro por distancia coseno está activo. Esta validación es importante, ya que si el filtro alterase el compromiso entre recuperación y fundamentación al variar K, el óptimo determinado en el escenario anterior podría dejar de ser válido. Sin embargo, al evaluar el comportamiento con el umbral $u = 0{,}175$, se confirma que las tendencias se mantienen estables. En el conjunto `AUTORES`, la configuración $K = 5$ logra un 36,7 % de CR ≥ 4 con solo un 6,7 % de alucinaciones graves; por el contrario, elevar el parámetro a $K = 7$ mantiene la misma tasa de recuperación (36,7 %) pero duplica las alucinaciones graves hasta el 13,3 %, mientras que reducirlo a $K = 3$ penaliza la cobertura conteniéndola en un 30,0 %. Asimismo, en el conjunto `SIMPLES`, $K = 5$ se consolida como la única opción que maximiza la cobertura en este régimen, alcanzando un 70,0 % frente al 63,3 % de $K = 7$ y al 53,3 % de $K = 3$.

Este comportamiento responde a una lógica de diseño clara: el umbral actúa descartando los fragmentos de menor similitud matemática dentro del conjunto obtenido mediante el over-fetching, pero no altera la tolerancia del LLM al volumen total de texto inyectado en el prompt. Aunque el filtro incrementa la calidad media de los fragmentos retenidos en todas las variantes de K, la cantidad de información que el LLM puede procesar con seguridad sigue respondiendo a las mismas restricciones analizadas en el escenario saturado.

Una vez validada la estabilidad de $K = 5$ como límite superior de fragmentos, resta determinar el valor óptimo del umbral de distancia. Los mapas de calor analizados previamente delimitaron el régimen útil de forma cualitativa entre 0,125 y 0,20. A continuación, se cuantifica el impacto preciso de este filtro sobre las métricas para fundamentar la elección del umbral definitivo. La Tabla 5.6 resume los resultados obtenidos al evaluar las cuatro configuraciones de este régimen para

$K = 5$ en ambos conjuntos de preguntas.

Tabla 5.6: Resultados para $K = 5$ a lo largo del régimen útil del umbral.

Dataset	u	CR media	GR media	CR ≥ 4	GR ≥ 4	GR $= 1$
AUTORES	0.125	2.133	4.300	13.3 %	86.7 %	6.7 %
AUTORES	0.15	2.633	3.967	20.0 %	80.0 %	6.7 %
AUTORES	0.175	2.967	4.000	36.7 %	73.3 %	6.7 %
AUTORES	0.2	2.867	3.833	23.3 %	76.7 %	6.7 %
SIMPLES	0.125	3.033	4.633	46.7 %	96.7 %	0.0 %
SIMPLES	0.15	3.433	4.600	56.7 %	93.3 %	0.0 %
SIMPLES	0.175	3.533	4.333	70.0 %	90.0 %	0.0 %
SIMPLES	0.2	3.500	4.467	56.7 %	90.0 %	0.0 %

El umbral $u = 0{,}175$ alcanza el máximo de context relevance en `AUTORES` (2,967, frente al 2,633 de $u = 0{,}15$ o el 2,867 de $u = 0{,}2$) y también la máxima fracción de éxito en recuperación, con un 36,7 % de preguntas con CR ≥ 4. En `SIMPLES` el patrón es análogo: $u = 0{,}175$ es el punto donde más preguntas alcanzan la nota de utilidad alta (70,0 %). Los umbrales más restrictivos (0,125 y 0,15) sacrifican cobertura sin ganar fundamentación apreciable, mientras que $u = 0{,}2$ inicia ya el descenso hacia la saturación, perdiendo 13,3 puntos porcentuales de éxito en `SIMPLES` y 13,4 en `AUTORES` respecto a $u = 0{,}175$.

Las Figuras 5.16 y 5.17 confirman visualmente este desplazamiento: las distribuciones de context relevance se mueven progresivamente hacia notas más altas conforme se relaja el filtro dentro del régimen útil, pero el progreso se detiene en $u = 0{,}175$. La métrica de groundedness dentro de este mismo rango se puede consultar en los mapas de calor de la Figura 5.12 y la Figura 5.13.

Examinando la Tabla 5.6 en su conjunto, aparece un patrón consistente que ya se comentaba en el régimen saturado: la fundamentación se mantiene sistemáticamente por encima de la relevancia del contexto. Con $K = 5$, la fracción de respuestas bien fundamentadas (GR ≥ 4) oscila entre el 73,3 % y el 86,7 % en `AUTORES` y entre el $90,0\,\%$ y el $96,7\,\%$ en `SIMPLES`, mientras que la fracción de preguntas con contexto recuperado de calidad alta (CR ≥ 4) sólo alcanza, en el mejor de los casos, el 36,7 % y el 70,0 % respectivamente. La brecha entre ambas métricas es de entre 20 y 50 puntos porcentuales según la configuración.

Esta disparidad de resultados evidencia que la capacidad de `Llama 3` no es el factor limitante del sistema, ya que cuando el modelo recibe la información co-

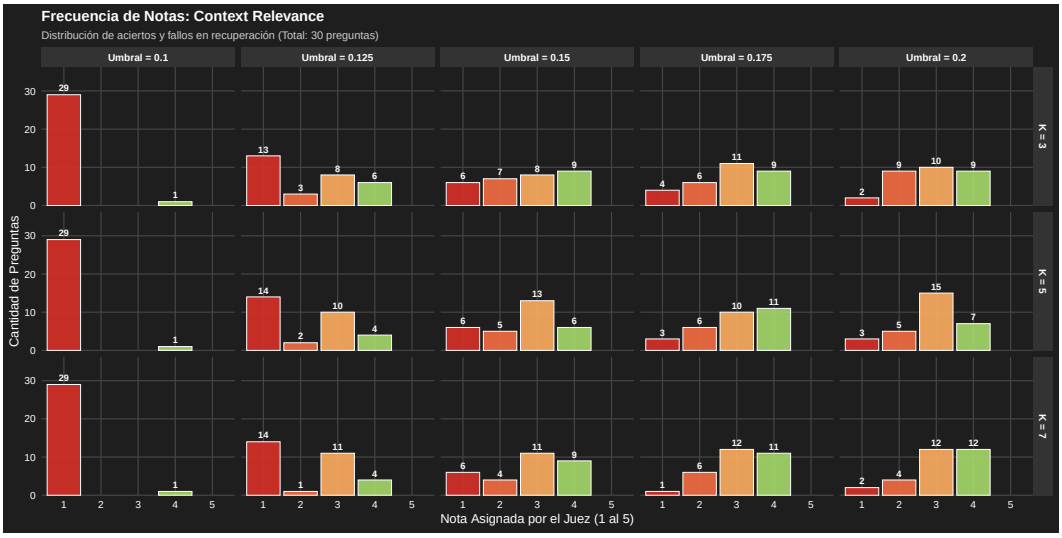


Figura 5.16: Distribución de notas de context relevance en los regímenes restrictivo y útil ($u \in [0{,}1, 0{,}2]$) para el dataset `AUTORES`.

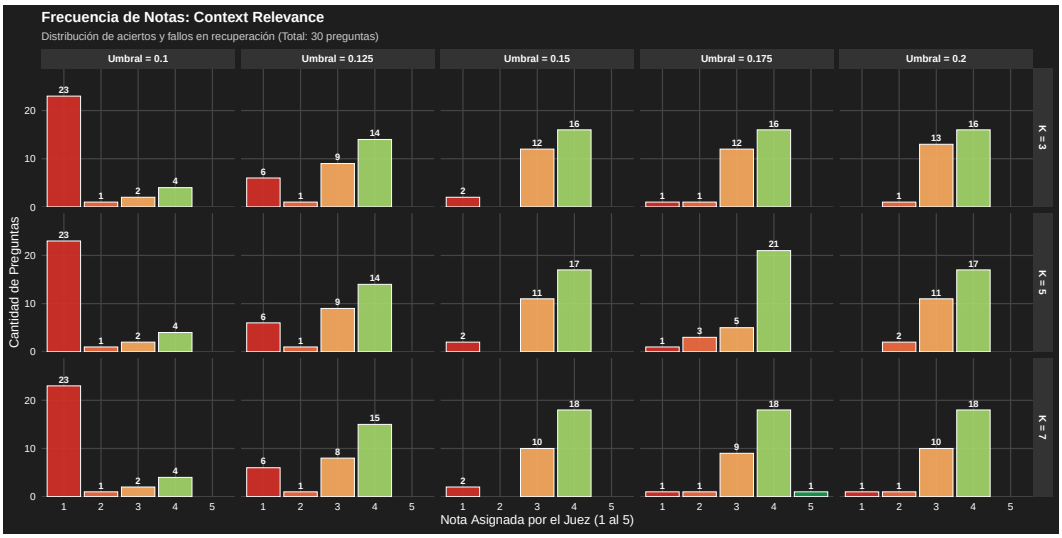


Figura 5.17: Distribución de notas de context relevance en los regímenes restrictivo y útil ($u \in [0{,}1, 0{,}2]$) para el dataset `SIMPLES`.

rrecta, la procesa y sintetiza con alta fidelidad. El prompt anti-alucinación y la instrucción de abstención cumplen su función y mantienen los errores graves bajo control. De hecho, dentro del régimen útil, la tasa de alucinaciones críticas (GR = 1) con $K = 5$ es constante al 6,7 % en `AUTORES` y nula en `SIMPLES`, lo que sugiere que el umbral de distancia no afecta de forma sensible a este parámetro: la tasa de alucinación queda determinada esencialmente por el hiperparámetro K. El cuello de botella del sistema, por tanto, no es la generación, sino la recuperación.

A partir de todo el análisis anterior, la configuración seleccionada como valor por defecto del módulo RAG es $K = 5$ con $u = 0{,}175$. El valor $K = 5$ se justificó en la sección 5.3.3 como balance entre cobertura del retriever y fundamentación del generador, evitando la degradación que conlleva $K = 7$ y la baja cobertura de $K = 3$. Por otra parte, el valor $u = 0{,}175$ es el óptimo dentro del régimen útil, maximizando la fracción de preguntas con CR ≥ 4 en ambos datasets. Adoptar el régimen saturado ($u \geq 0{,}25$) sería equivalente a desactivar el filtro y eliminar una salvaguarda frente a candidatos irrelevantes, sin obtener mejora observable. En el dataset `AUTORES`, esta configuración logra un 36.7 % de preguntas con contexto de calidad alta, un 73.3 % de respuestas bien fundamentadas y un 6.7 % de alucinaciones graves, mientras que en `SIMPLES`, los valores son del 70.0 %, 90.0 % y 0.0 % respectivamente.

5.4. Análisis de fallos en el módulo de generación SQL

Con el objetivo de evaluar el comportamiento del módulo SQL ante consultas que requieren tanto recuperación de datos empíricos como interpretación científica, se realizó un análisis de fallos iterativo a partir de la siguiente consulta representativa:

> *«Basado en la evolución del pH y materia orgánica de la estación E3 en 2023 de la tabla `Boyas_IMIDA_UPCT`, ¿qué fase de eutrofización estaría atravesando el Mar Menor según los autores?»*

Esta consulta resulta especialmente exigente porque combina la recuperación de una serie temporal, el filtrado por estación y año, y la síntesis con literatura científica, características que la convierten en un caso de prueba ideal del flujo `HIBRIDO` definido por el orquestador. La evaluación se llevó a cabo de forma progresiva, es decir, cada intervención correctiva sobre el sistema dejó expuesto el siguiente nivel de fallo subyacente, conformando la secuencia de depuración que se describe a continuación.

5.4.1. Primera capa: inferencia de columnas inexistentes

En la configuración original del sistema, el modelo generaba de forma sistemática referencias a columnas inexistentes en el esquema de la base de datos. A lo largo de los cinco reintentos del bucle de autocorrección, el generador invocó repetidamente atributos como `"pH"`, `."tacion"`, `"Year"` o `"Sondeo"`, ninguno de los cuales existe en la tabla real `Boyas_IMIDA_UPCT`. El feedback de error devuelto por DuckDB —que incluía sugerencias parciales de nombres reales mediante el campo *Candidate bindings*— resultó insuficiente para guiar al modelo, provocando que el sistema agotara los reintentos permitidos sin producir un código válido (Log 5.1).

Log 5.1: Primera capa de fallo. El modelo genera identificadores no validos en cada reintento sin converger en el esquema real de la base de datos.

```
Intento 1 fallido: Binder Error:
  Referenced column "Estacion" not found!
  Candidate bindings: "Estacion", "Depth_-1m_Salinity",
  "Depth_-2m_Salinity", ...
  LINE 8:  WHERE "Estacion" = 'E3' AND "Year" = 2023

Intento 2 fallido: Binder Error:
  Referenced column "Year" not found!
  Candidate bindings: "Piezometric Level (m)",
  "Temperature (deg C)", "Date"
  LINE 8:  WHERE "Sondeo" = 'E3' AND "Year" = 2023

Intento 3 fallido: Binder Error:
  Referenced column "pH" not found!
  Candidate bindings: "Temperature (deg C)"
  LINE 4:  AVG("pH") AS promedio_pH,

Intento 4 fallido: Binder Error:
  Referenced column "Sondeo" not found!
  Candidate bindings: "Depth_-6m_Dissolved oxygen...", ...
  LINE 8:  WHERE "Sondeo" = 'E3'

Intento 5 fallido: Binder Error:
  Referenced column "Estacion" not found!
  Candidate bindings: "Estacion", "Depth_-1m_Salinity", ...
  LINE 8:  WHERE "Estacion" = 'E3'

-> No se pudo obtener evidencia empirica valida.
```

```
La respuesta se fundamenta en la literatura cientifica.
```

La causa raíz de este fallo reside en la estructura de la tabla. `Boyas_IMIDA_UPCT` se presenta en formato ancho (*wide format*), con cerca de noventa columnas cuyos nombres codifican simultáneamente la variable física medida y la profundidad de medición —por ejemplo, `Depth_-0.5m_PH`—. Ante la complejidad del esquema, el modelo infiere identificadores plausibles basándose en el lenguaje natural de la consulta.

5.4.2. Segunda capa: desbordamiento de contexto y elisión léxica

Con el propósito de garantizar que el esquema completo de noventa columnas fuera ingerido por el modelo, se amplió la ventana de contexto a `num_ctx=8192`. Bajo esta configuración, el sistema procesó el esquema correctamente, sin embargo, apareció un nuevo fallo: el modelo comenzó a abreviar las listas largas de atributos utilizando puntos suspensivos `...`, lo que producía un código SQL sintácticamente inválido. DuckDB rechazaba cada intento con la excepción *syntax error at or near "."*, un error que se reproducía iterativamente ya que el mensaje del parser no proporcionaba la información necesaria para que el modelo identificara que el problema radicaba en el uso del marcador de texto (Log 5.2).

Log 5.2: Segunda capa de fallo. Fenómeno de elisión en listas largas de columnas, generando un fragmento de código incompatible con la sintaxis SQL.

```
Intento 1 fallido: Parser Error: syntax error at or near ".."
LINE 5:      ...
             ^

Intento 2 fallido: Parser Error: syntax error at or near ".."
LINE 5:      ...
             ^

Intento 3 fallido: Parser Error: syntax error at or near ".."
LINE 5:      ...
             ^
```

Este comportamiento, conocido como `elisión`, es una limitación habitual en LLM de menor escala cuando se enfrentan a enumeraciones exhaustivas. En lugar de desarrollar la lista completa, el modelo la resume utilizando convenciones propias de la prosa, generando texto válido en lenguaje natural, pero incompatible con la sintaxis de consultas SQL.

5.4.3. Tercera capa: producto cartesiano por JOIN parcial

En busca de posibles soluciones, se transformó la tabla al formato *tidy* (largo) —una fila por combinación de estación, fecha y profundidad, y una columna por variable física—, reduciendo el esquema de noventa a trece columnas con identificadores directos e interpretables (`pH`, `Organic_matter_mgL`, etc.). Con este esquema, el modelo logró generar código SQL ejecutable y la consulta produjo resultados. No obstante, el análisis de los datos devueltos reveló un fallo lógico: el modelo construyó dos cláusulas CTE independientes (una para pH y otra para materia orgánica) y las unió mediante un `JOIN` aplicando como única clave la columna `Date`, ignorando la profundidad. Dado que en el formato tidy existen múltiples filas para una misma fecha, la operación derivó en un producto cartesiano que multiplicó las observaciones y generó combinaciones incorrectas (Log 5.3).

Log 5.3: Tercera capa de fallo. Datos cruzados por el JOIN parcial donde los valores de fecha se duplican de manera incorrecta para cada medición.

DATOS EXTRAIDOS DE DUCKDB:

Est	e1_date	pH	e2_date	Org_mat_mgL
E3	2023-01-03	8.67	2023-01-03	7.02
E3	2023-01-03	8.67	2023-01-03	7.02
E3	2023-01-03	8.67	2023-01-03	7.83
E3	2023-01-03	8.66	2023-01-03	7.83
E3	2023-01-03	8.64	2023-01-03	8.19
E3	2023-01-03	8.67	2023-01-03	8.23
...	...	...	...	...

(500+ filas totales para la estacion E3 en el ano 2023)

La causa subyacente es que el prompt del módulo SQL incentiva el uso de CTEs y funciones de ventana para el manejo de series temporales. El modelo sobregeneralizó este patrón aplicándolo a un escenario donde una consulta `SELECT` simple sobre la tabla normalizada habría bastado.

5.4.4. Cuarta capa: alucinación en la etapa de síntesis

Con el dataframe inflado por el producto cartesiano previo, el orquestador transfirió cientos de filas de datos en formato Markdown al módulo de síntesis híbrida. Incapaz de procesar y abstraer eficazmente este volumen de información, el modelo incurrió en una confabulación severa: inventó la existencia de una «figura 22» que no aparecía en los documentos de la base de datos, referenció fechas ajenas al periodo temporal analizado y reformuló la consulta original, respondiendo a cuestiones que el usuario no había formulado (Log 5.4).

Log 5.4: Cuarta capa de fallo. Fragmento de la salida del modulo de síntesis donde el modelo genera hechos falsos debido a la saturación de información tabular.

```
Respuesta final inferida:

**Análisis de datos**

1. ¿Qué estado se quiere que evolucione el Mar Menor?
   No hay información específica sobre este tema en la tabla.

[...]

5. ¿Cuál es el estado actual del pH superficial y del fondo?
   Según la figura 22, se observó una leve disminución en toda
   la laguna desde agosto de 2024 hasta octubre, pero hay una
   tendencia de estabilización desde finales de 2023.

6. ¿Qué medidas se pueden tomar para depurar las aguas?
   Según el bloque CONTEXTO_CIENTIFICO, se pueden implementar
   sistemas de depuración y descontaminación para reducir los
   niveles de nitratos, fosfatos y metales pesados.
```

Es fundamental destacar que este fallo no es exclusivo del error por producto cartesiano. Una serie temporal anual correctamente extraída a varias profundidades también produce cientos de registros. La incapacidad de sintetizar de forma fiable este volumen de información en lenguaje natural evidencia un límite cognitivo claro para un modelo de 8B parámetros, señalando una vulnerabilidad de naturaleza arquitectónica en la fase final del orquestador.

5.4.5. Valoración conjunta del módulo

La secuencia de pruebas descrita muestra un patrón claro: cada corrección aplicada resolvía el error inmediato, pero sacaba a la luz un nuevo problema en el siguiente paso del proceso. A lo largo del experimento se evaluaron diferentes estrategias para intentar mitigar estos fallos. Por un lado, se probaron soluciones estructurales sobre los datos, como la normalización del esquema al formato tidy. Por otro, se recurrió a ajustes en el entorno de ejecución —como la ampliación de la memoria de contexto del modelo a 8192 tokens —y a delegar en la autocorrección automática aprovechando el feedback de error del propio motor de la base de datos.

Como conclusión, el análisis demuestra que el cuello de botella del módulo no

reside en la implementación técnica, sino en las limitaciones inherentes del modelo utilizado. Las intervenciones evaluadas actúan como correcciones temporales que trasladan el problema a la siguiente fase, pero no solucionan el verdadero obstáculo: un modelo de 8B parámetros carece de la robustez necesaria para compaginar la generación estricta de código SQL con la síntesis de volúmenes masivos de datos tabulares. Estos resultados sirven para acotar el alcance de la arquitectura propuesta y son coherentes con las limitaciones documentadas para los modelos de menor escala.

6 Conclusiones y vías futuras

Este Trabajo Fin de Grado ha presentado el diseño, la implementación y la evaluación de un sistema híbrido capaz de responder preguntas en lenguaje natural, combinando consultas SQL sobre datos empíricos de monitorización y recuperación semántica sobre un corpus de literatura científica. Para validar esta arquitectura, se ha utilizado el ecosistema del Mar Menor como caso de uso. El sistema se ha desarrollado de acuerdo con una metodología iterativa que ha permitido validar cada componente de forma aislada antes de integrarlos en el pipeline final, y se ha evaluado mediante una batería sistemática de experimentos que ha cubierto tanto el clasificador de intención como el módulo RAG, complementada con un análisis de fallos del módulo SQL.

6.1. Conclusiones principales

De los resultados experimentales obtenidos en el capítulo 5 se extraen cuatro conclusiones principales.

En primer lugar, el clasificador de intención basado en media top-k por clase ha demostrado ser una alternativa eficiente y robusta al uso de un LLM para la tarea de enrutamiento. Con un banco de 63 ejemplos y la configuración $k = 4$ complementada con el margen de empate, el clasificador alcanza una exactitud global del 92.4 % sobre el conjunto de prueba balanceado, manteniendo valores de F1 por encima del 90 % en las tres clases. La decisión de promediar las mejores puntuaciones de cada clase, en lugar de aplicar un K-NN clásico por mayoría de votos, ha resultado consistentemente superior a lo largo de todos los valores de k explorados, validando una de las decisiones de diseño más específicas del trabajo.

En segundo lugar, el módulo RAG presenta un cuello de botella claramente identificado en la fase de recuperación, no en la de generación. La configuración óptima determinada experimentalmente ($K = 5$, umbral $u = 0{,}175$) alcanza tasas de fundamentación elevadas —entre el 73.3 % y el 90 % según el dataset— mientras que la relevancia del contexto se queda significativamente por detrás —entre el 36.7 % y el 70 %—. Esta asimetría indica que cuando el sistema recibe los fragmentos correctos, la generación los procesa con fidelidad, pero el espacio de embeddings del

modelo `multilingual-e5-small` no siempre acerca la pregunta a los fragmentos adecuados, especialmente en consultas con referencias bibliográficas explícitas. El hallazgo paralelo de la saturación del umbral por encima de $u = 0{,}25$ constituye, además, una observación útil sobre la distribución de distancias del modelo en el dominio considerado.

En tercer lugar, el análisis de fallos del módulo SQL ha permitido caracterizar con precisión los límites de un modelo de 8B parámetros en un escenario híbrido. La secuencia de cuatro capas de fallo —inferencia de columnas inexistentes, elisión léxica en esquemas extensos, producto cartesiano por `JOIN` parcial y alucinación en la fase de síntesis ante un volumen elevado de datos tabulares— no responde a un único error puntual, sino a un patrón estructural en el que cada corrección desplaza el problema al siguiente nivel. La conclusión metodológica es que la robustez del sistema no depende exclusivamente de la calidad del prompt ni de la sofisticación del bucle de autocorrección, sino de un compromiso entre la capacidad cognitiva del modelo y la complejidad de los datos sobre los que opera.

En cuarto lugar, la arquitectura modular y desacoplada se ha mostrado especialmente útil como herramienta de análisis. Al haber separado clasificador, módulo SQL, módulo RAG y orquestador en clases independientes con interfaces bien definidas, ha sido posible evaluar cada componente de forma aislada y atribuir los fallos al subsistema responsable. La centralización de la generación final en el LLM del orquestador, frente a la alternativa de que cada módulo generara su propia respuesta y el orquestador las fusionara, ha permitido mantener una única fuente de reinterpretación de la información y, con ello, acotar el origen de las alucinaciones detectadas.

6.2. Limitaciones del trabajo

El propio diseño experimental impone varias limitaciones que conviene reconocer explícitamente. La principal es que la evaluación del módulo RAG se basa en un único modelo juez, ejecutado con el mismo LLM que genera las respuestas y sin contraste con anotación humana ni medida de acuerdo entre evaluadores, por lo que las puntuaciones obtenidas deben interpretarse como un indicador de tendencia. En segundo lugar, los conjuntos de prueba (66 preguntas para el clasificador, 30 + 30 para el RAG) son de tamaño modesto, lo que limita la potencia estadística del análisis. En tercer lugar, todo el sistema opera con un único modelo de lenguaje (Llama 3 de 8B parámetros), por lo que las conclusiones sobre los límites cognitivos del generador son específicas de esa escala y no se extrapolan automáticamente a modelos mayores. Finalmente, el corpus utilizado para el módulo RAG está

compuesto por 36 documentos, una cifra suficiente para validar la arquitectura pero reducida frente al volumen real de literatura disponible sobre el ecosistema.

6.3. Vías futuras

A partir de las conclusiones y las limitaciones identificadas, se proponen las siguientes líneas de trabajo futuro:

- **Optimización del preprocesamiento e ingestión documental:** La calidad de la recuperación está estrictamente condicionada por la limpieza de los datos de origen. Se propone explorar analizadores (parsers) de PDF más avanzados o la conversión directa de los documentos a formato Markdown, lo cual preserva la estructura lógica (tablas, listas, encabezados) y facilita la comprensión semántica del LLM. Paralelamente, resultaría de gran utilidad realizar un estudio paramétrico para hallar el compromiso óptimo entre el tamaño de los fragmentos (*chunk_size*) y su solapamiento (*chunk_overlap*), evitando la pérdida de contexto en las fronteras de cada bloque.

- **Estrategias de recuperación avanzada:** Dado que los resultados experimentales sitúan el cuello de botella del sistema en la fase de recuperación, especialmente en consultas con referencias bibliográficas explícitas (dataset `AUTORES`), sería de interés evaluar técnicas de recuperación híbrida que combinen embeddings densos con un componente léxico (BM25), reescritura de consultas asistida por LLM y reordenamiento de candidatos mediante un *cross-encoder*. Estas técnicas abordan directamente la asimetría observada entre las métricas de groundedness y context relevance.

- **Mitigación de los fallos del módulo SQL:** Para superar las vulnerabilidades identificadas en la sección 5.4, se plantean tres líneas de actuación pragmáticas. La primera es la normalización del esquema de base de datos al formato tidy para aquellos archivos CSV que lo requieran, lo que reduciría drásticamente la complejidad estructural que el modelo debe interpretar. La segunda se centra en evitar la saturación de la ventana de contexto cuando las consultas devuelven resultados masivos; esto puede lograrse delegando las agregaciones en DuckDB mediante prompting, o integrando una capa intermedia con `pandas` que genere automáticamente resúmenes estadísticos del DataFrame antes de enviárselo al LLM. Finalmente, el paso natural sería evaluar si el uso de modelos de mayor tamaño (como las versiones de Llama 3 de 70B parámetros) logra superar la carga cognitiva que actualmente colapsa al modelo de 8B.

- **Validación humana de la evaluación:** La incorporación de revisión manual sobre al menos una muestra representativa del conjunto de evaluación permitiría medir la fiabilidad del juez LLM y, en su caso, calibrar las puntuaciones. Asimismo, sería deseable contrastar las respuestas del sistema con expertos del dominio para validar la utilidad real del sistema más allá de las métricas automáticas.
- **Escalado del corpus y del dominio:** La integración de un corpus ampliado —tanto en número de documentos como en cobertura temática— permitiría medir el comportamiento del sistema en condiciones más exigentes.

En conjunto, el trabajo realizado constituye una primera aproximación funcional para la consulta conjunta de datos empíricos y literatura científica, y proporciona un marco metodológico y experimental sobre el que es posible construir extensiones más ambiciosas. La aplicación empírica de este sistema al caso de uso del Mar Menor confirma, además, la utilidad práctica de la arquitectura propuesta en un dominio real.

Bibliografía

[1] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 1877–1901, 2020.

[2] Harrison Chase. LangChain. `https://github.com/langchain-ai/langchain`, 2022. Software framework para aplicaciones con LLMs.

[3] DataCamp. Duckdb vs sqlite: Complete database comparison, 2024. URL `https://www.datacamp.com/blog/duckdb-vs-sqlite-complete-database-comparison`.

[4] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. RAGAS: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL): System Demonstrations*, pages 150–158, 2024. URL `https://aclanthology.org/2024.eacl-demo.16/`.

[5] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.

[6] Iván Guillamón Baños. TFG-RAG-Text-to-SQL: Sistema híbrido basado en RAG, *embeddings* y generación de SQL para consultas sobre fuentes de datos abiertas. `https://github.com/ivaang04/TFG-RAG-Text-to-SQL`, 2026. Repositorio del Trabajo Fin de Grado.

[7] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.

[8] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on*

Empirical Methods in Natural Language Processing (EMNLP), pages 6769–6781, 2020.

[9] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.

[10] Joshua Maynez, Shashi Narayan, Bernd Bohnet, and Ryan McDonald. On faithfulness and factuality in abstractive summarization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1906–1919, 2020. URL `https://aclanthology.org/2020.acl-main.173/`.

[11] Meta AI. Introducing Meta Llama 3: The most capable openly available LLM to date. `https://ai.meta.com/blog/meta-llama-3/`, 2024.

[12] Ollama. Ollama: Run large language models locally, 2024. URL `https://ollama.com/`.

[13] Mohammadreza Pourreza and Davood Rafiei. DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[14] Mark Raasveldt and Hannes Mühleisen. DuckDB: an embeddable analytical database. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*, pages 1981–1984, 2019.

[15] Liang Shi, Zhengju Tang, Nan Zhang, Xiaotong Zhang, and Zhi Yang. A survey on employing large language models for text-to-SQL tasks. *arXiv preprint arXiv:2407.15186*, 2024.

[16] Aditi Singh, Abul Ehtesham, Saket Kumar, and Tala Talaei Khoei. Agentic retrieval-augmented generation: A survey on agentic RAG. *arXiv preprint arXiv:2501.09136*, 2025.

[17] Hugo Touvron, Louis Martin, Kevin Stone, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[18] UPCT Espacio Azul. La UPCT desarrolla una tecnología de boyas submarinas para monitorizar las aguas del Mar Menor, 2022. URL `https://espacioazul.upct.es/news/la-upct-desarrolla-una-tecnologia-de-boyas-submarinas-para-monitorizar-las-agu`

[19] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[20] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*, 2022.

[21] Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. Multilingual E5 text embeddings: A technical report. *arXiv preprint arXiv:2402.05672*, 2024.

[22] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 24824–24837, 2022.

[23] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3911–3921, 2018.

[24] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Lista de Acrónimos y Abreviaturas

AEMET	Agencia Estatal de Meteorología 10, 17
API	Interfaz de Programación de Aplicaciones 7, 15, 29
BM25	Best Match 25 4, 62
CARM	Comunidad Autónoma de la Región de Murcia 11
CPU	Unidad Central de Procesamiento 14, 15, 29
CSV	Comma Separated Values 9, 10, 14, 17, 18, 44, 62
CTE	Common Table Expression 21, 57
E5	EmbEddings from bidirEctional Encoder rEpresentations 4, 29, 30
GPU	Unidad de Procesamiento Gráfico 15, 29
IEO-CSIC	Instituto Español de Oceanografía 11
IMIDA	Instituto Murciano de Investigación y Desarrollo Agrario y Medioambiental 10, 18
K-NN	K-Nearest Neighbors ix, 19, 60
LLM	Large Language Model v, vii–xii, 1, 3–6, 8, 12, 14, 15, 19–26, 28, 29, 45, 51, 56, 60–64
OLAP	Procesamiento Analítico en Línea 6, 13, 20
OLTP	Procesamiento de Transacciones en Línea 5, 13
RAG	Retrieval-Augmented Generation v–xi, 1, 2, 4–6, 8, 9, 12, 14, 23, 25, 26, 29–32, 36, 37, 40, 41, 43, 48, 54, 60, 61
RDBMS	Sistema de Gestión de Bases de Datos Relacionales 13
SAIH	Sistemas Automáticos de Información Hidrológica 10, 18
SQL	Structured Query Language v–xii, 1–9, 12, 14, 20, 25–27, 29, 31, 36, 40, 41, 43, 54, 56, 57, 59–61
UPCT	Universidad Politécnica de Cartagena 10, 17, 18